O'REILLY

Snowflake Billing Model

Tomas Sobotik

Learning objectives

By the end of this live, hands-on course, you'll understand:

- Common performance issues and how to solve them
- Strategies for performance optimization
- Proficient Data Loading
- Effective Virtual Warehouse configuration
- Query Life Cycle
- Table clustering and micro partition pruning

.

Tomas Sobotik

- Based in Czech Republic
- 2 kids
- 15+ years in data industry
- Data engineer and architect
- Big Snowflake fan and enthusiast
- Snowflake Data Superhero
- Snowflake Subject Matter Expert

Cost drivers

- SaaS solution - no license or HW to buy
- Usage based pricing
- Billing is done per layer - storage, compute, cloud services
- Base invoiceable unit is called Snowflake Credit
  - Used to pay for the consumption of resources
  - Consumed only when some resource is used
- Consumption model - pay only for what you actually use
  - Storage = TB of disk space
  - Compute = running time of virtual warehouses

Credit price components

Snowflake Edition

Standard, Enterprise,
Business Critical and
Virtual Private Snowflake

Cloud Provider

aws

Google Cloud

Azure

Snowflake is cloud agnostic,
available on all major clouds

Region

Global presence with
cross region, cross cloud
replication
or data collaboration

Storage costs

- Based on average amount of terabyte storage used per month
- Calculation is based on compressed and ingested data into Snowflake
- Ingested data are always compressed, encrypted and columnrised
- 5x - 10x compression based on data types
- Calculated as daily average amount
- Storage costs includes
- Data itself
  - Time Travel Data
  - Fail Safe Data
  - Internal stages

Compute costs

- given by credits consumption
- warehouse size (number of nodes) corresponds to Snowflake credits for full hour of run time
  - XS Warehouse: 1 credit for 1 hour
- charged by second after first minute
- when suspended = no charge (auto suspend!)
- Usually 95% of all costs

	XS	S	M	L	XL	2XL	3XL	4XL	5XL*	6XL*
credits/hour	1	2	4	8	16	32	64	128	256	512

Cloud service layer costs

- authentication, metadata management, SQL API, Access Control, Query parsing and optimization
- Snowflake credits are also used for paying the usage of cloud service layer
- Free up to 10% of the daily usage of compute resources

Date	Credits Used	Cloud Services Credits Used	Credit Adjustment for Cloud Services	Credits Billed totally
Feb 21	100	20	-10	110
Feb 22	120	10	-10	120

What else?

- Serverless features
  - Snowpipes (1.25 credits per compute hour)
  - Serverless tasks (0.9x of tasks running on own managed warehouse)
  - Clustering (AUTOMATIC_CLUSTERING warehouse)
  - MV refresh (MATERIALIZED_VIEW_MAINTENANCE)
  - Search optimization service
  - …

Keep in mind

- some particular features can have significant impact on cost. Needs to be used wisely
  - materialized views
  - auto clustering
  - replication
- always keep cost in mind - it is Pay as you go service
- cost optimization might be important part in later phases of the projects
  - use warehouse parameters
  - utilize Snowflake caches
  - ingest data effectively
  - monitor your costs

Poll questions

1. How is compute billed in Snowflake
    A. Fixed amount per month
    B. Based on current usage
    C. Compute is free

3. How much do you need to always pay for running virtual warehouse
    A. 1 hour
    B. 1 minute
    C. 1 second

Ě. What is not Snowflake cost driver
    A. Number of Snowflake users
    B. Cloud provider
    C. Used region

O'REILLY

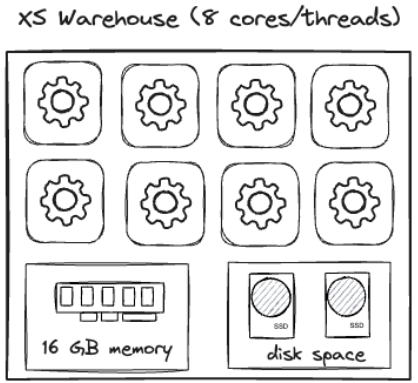
Virtual warehouse configuration

How to properly create compute cluster

Tomas Sobotik

Virtual Warehouses

- A named wrapper around a cluster of servers with CPU, memory, and disk
  - A compute cluster = logical collection of VMs (1+)
- The larger the warehouse, the more servers in the cluster
- Snowflake utilize T-shirt sizes for Virtual Warehouses
- Each size up doubles compute resources
  - Number of nodes correspond to configured size of VW
- Jobs requiring compute run on virtual warehouse
- While running, a virtual warehouse consumes Snowflake credits
- Snowflake account object



Warehouse sizes

Warehouse Size	Servers	Credits / Hours	Credits / Second
X-Small	1	1	0.003
Small	2	2	0.006
Medium	4	4	0.0011
Large	8	8	0.0022
X-Large	16	16	0.0044
2X-Large	32	32	0.0089
3X-Large	64	64	0.0178
4X-Large	128	128	0.0356
5X-Large*	256	256	0.0711
6x-Large*	512	512	0.1422

* Preview feature in some regions

Scale up for performance

Elastic processing power - CPU, RAM, SSD

Boost the performance for complex queries or ingesting large datasets

More complex queries on larger datasets require larger Warehouse

Concurrency issues (more users/queries) can't be solved by large warehouse

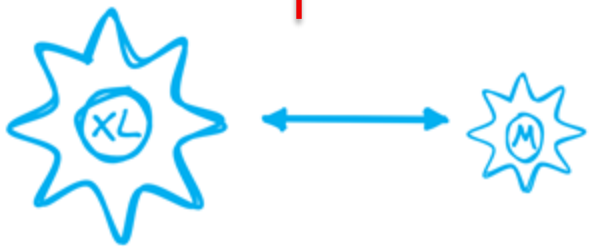
Guidelines for warehouse sizing

Utilize the per second billing

Use larger warehouse for more complex queries, workloads and suspend them when they are not in use

Experiment with warehouse sizes to find the right balance between performance and cost

Keep similar queries on the same warehouse to simplify compute resource sizing



Scale out for concurrency

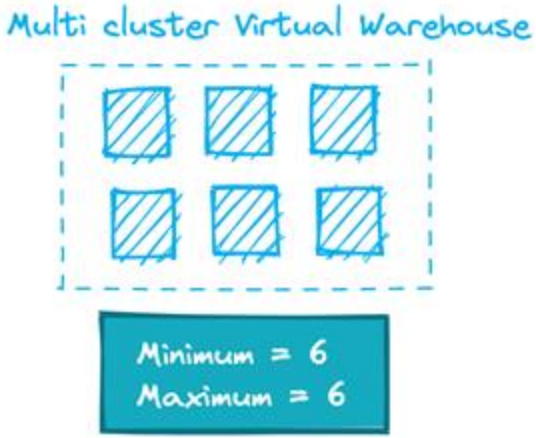
Multi Cluster Warehouse

- Single virtual warehouse with multiple compute clusters
- Scale out during peak times (more users, concurrent queries) and scale back when additional power is not needed
- Additional resources could be allocated statically or dynamically -> scaling modes & policies
- Queries are distributed across the clusters in a Virtual Warehouse
- To support high availability clusters are deployed across availability zones

Properties

```
MIN_CLUSTER_COUNT: 1 - 10 (default 1)
MAX_CLUSTER_COUNT: 1 - 10 (default 1)

MAX_CLUSTER_COUNT >=
MIN_CLUSTER_COUNT
```



Basic warehouses operations

- Start / Stop
  - could be automated via warehouse parameters
- AUTO_SUSPEND
  - warehouse will be suspended after specified period of time
  - **enabled by default**
- AUTO_RESUME
  - warehouse is automatically resumed when any statement that requires a warehouse is submitted
  - current session needs to have specified warehouse
  - **enabled by default**
- Resizing
  - could be done anytime, even when WH is running and queries are being executed
  - ALTER WAREHOUSE Command or via UI
  - Effects
    - Suspended warehouse will start at new size when it will be resumed
    - Running warehouse: immediate impact
      - running queries - finish with current size
      - new queries run at new size

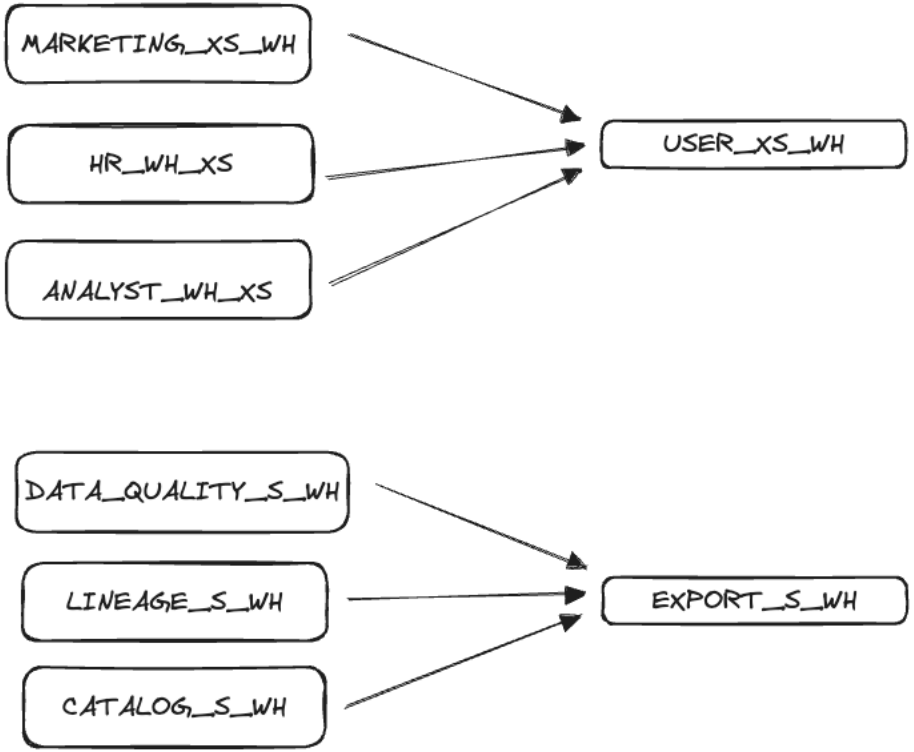
Best practices

- Start with an X-SMALL size
  - Increasing the size until query duration stops halving -> not fully utilizing wh anymore
- Set max_cluster_count based on peak concurrency or your SLA
- 60s auto-suspend then modify per use case (cache impact)
- Configure query timeouts (default is 2 days)
- Configure alerts on usage spikes (resource monitors)
- If queuing is acceptable reduce the warehouse size (i.e. data loading)
- Separate whs by workload requirements, not domain
- Try to keep warehouse fully loaded
  - Can run multiple queries at same time
  - Light queuing might be for non-user warehouses

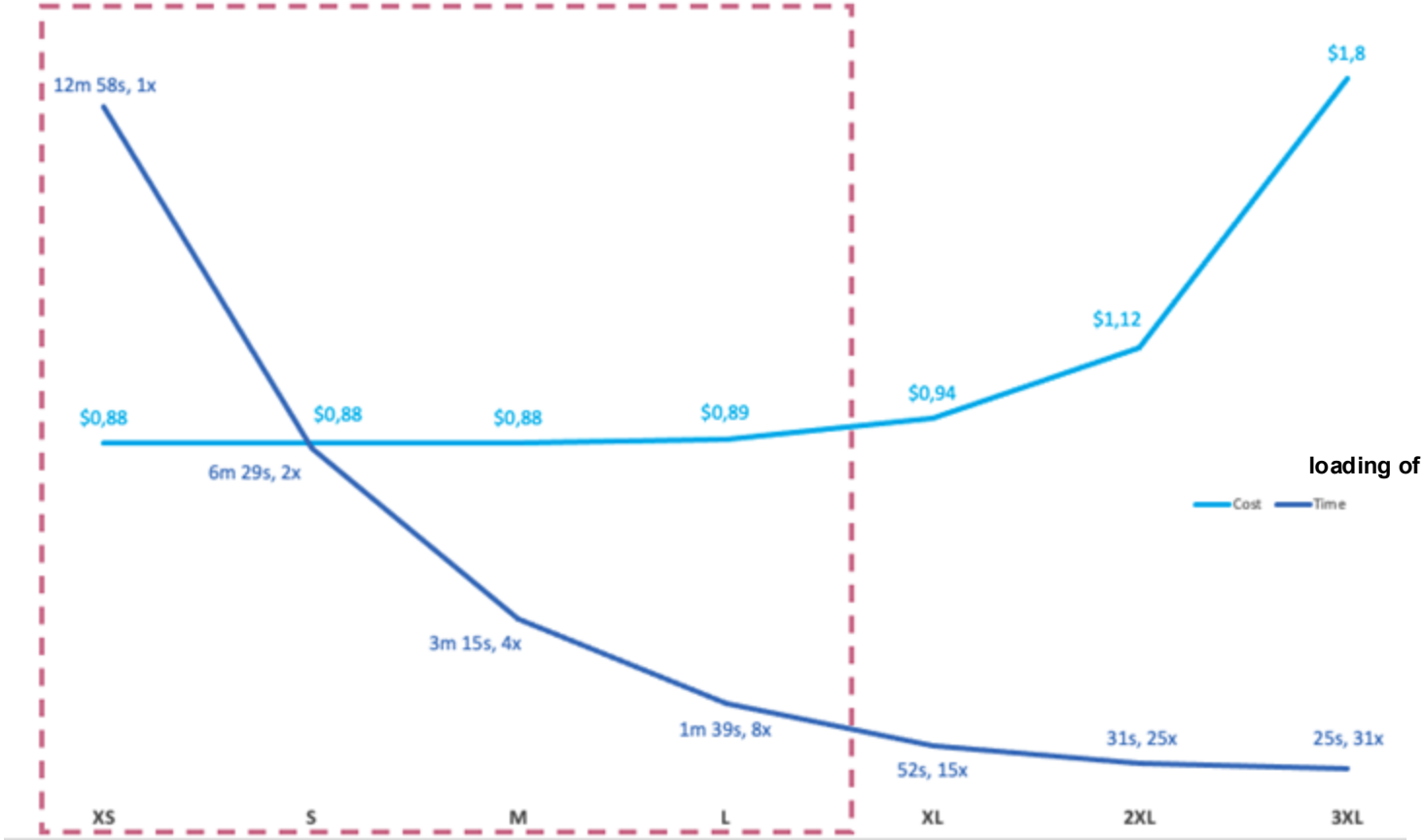
Avoid spilling

Warehouse consolidation

- Fewer warehouses -> less idle time
- Speed up the queries because of caching



Bigger does not mean more expensive



Exercise

We are going to practice the creation of virtual warehouses (single, multi cluster one) with different setup. Post creation we will be modifying a few characteristics to avoid unintended cost – e. g. query timeouts, resource monitors.

Detailed instructions could be found on **github**

O'REILLY

Query plan

How to work with it

Tomas Sobotik

Query Plan / Query Profile

- DAG consisting of operators connected by links
- Operators – process a set of rows
  - Table scan, filter, join, order, aggregate, etc.
- Links
  - Exchanging data between operators
  - Handle parallel redistribution of data at run time
- Detailed statistics related to each operator and whole query
- Helps understand how well query performed
- Entry point for performance optimization of individual queries
- It helps you identify performance bottlenecks or spot typical mistakes in SQL query expression

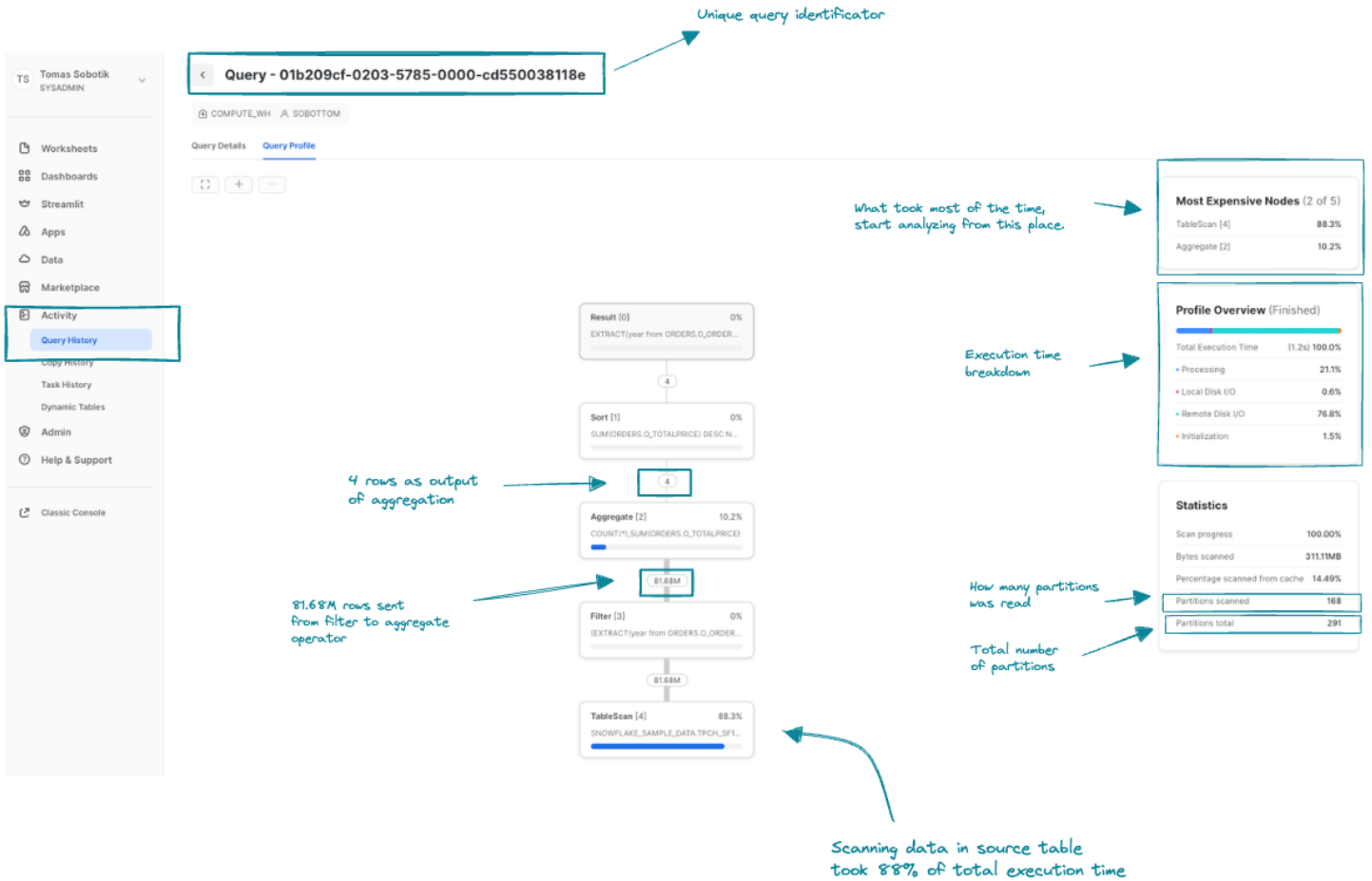
How to access it?

- Query history
- Query Details in Worksheet

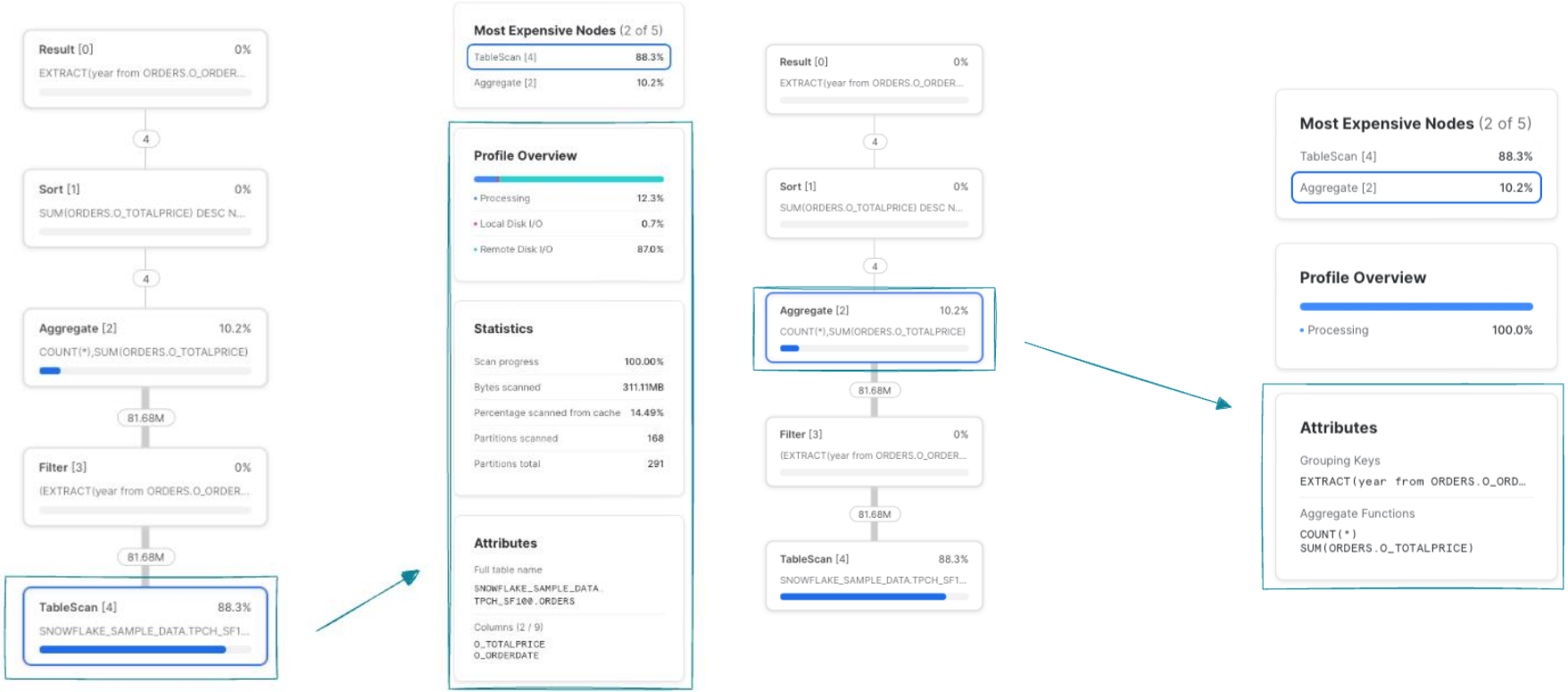
Query profile deep dive

```
select
 year(o_orderdate) as year,
 count(*) order_count,
 sum(o_totalprice) total_amount
from orders
where year(o_orderdate) > 1994
group by year
order by 3 desc;
```

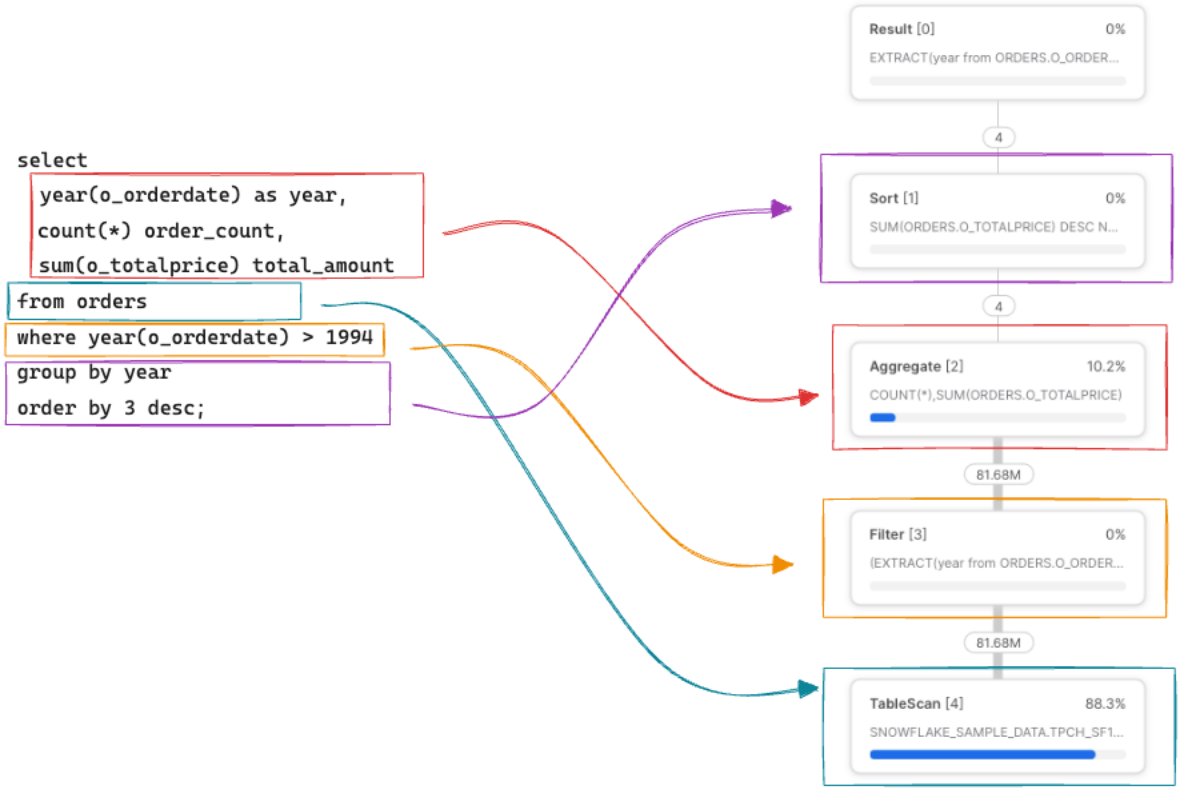
Query profile deep dive



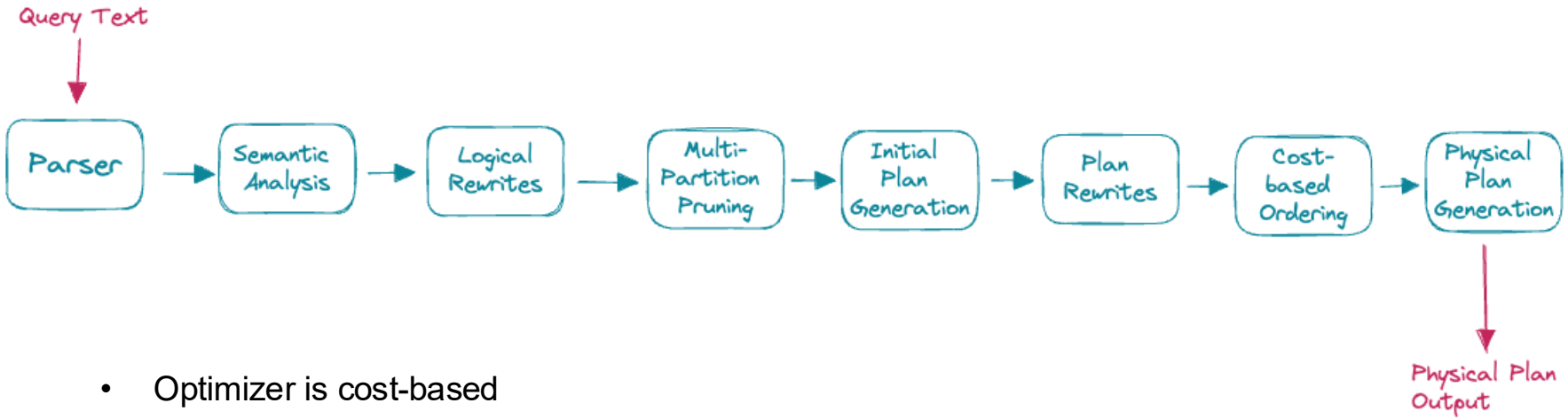
Operator details



Query mapping to operators



Query compilation details



- Optimizer is cost-based
- Leave as much to execution as possible
- Leave as little to the user as possible
- Focus on optimization for analytics queries
  - Support any data model

Working with GET_QUERY_OPERATOR_STATS function

```
select * from table(get_query_operator_stats('01b209cf-0203-5785-0000-cd550038118e'));
```

	QUERY_ID	STEP_ID	OPERATOR_ID	PARENT_OPERATORS	OPERATOR_TYPE	OPERATOR_STATISTICS
1	01b209cf-0203-5785-0000-cd550038118e	1	0	null	Result	{ "input_rows": 4 }
2	01b209cf-0203-5785-0000-cd550038118e	1	1	[0]	Sort	{ "input_rows": 4, "output_rows": 4 }
3	01b209cf-0203-5785-0000-cd550038118e	1	2	[1]	Aggregate	{ "input_rows": 81677302, "output_rows": 4 }
4	01b209cf-0203-5785-0000-cd550038118e	1	3	[2]	Filter	{ "input_rows": 81677302, "output_rows": 81677302 }
5	01b209cf-0203-5785-0000-cd550038118e	1	4	[3]	TableScan	{ "io": { "bytes_scanned": 326220032, "percentage_scanned_fr


```
{
  "io": {
    "bytes_scanned": 326220032,
    "percentage_scanned_from_cache": 0.1449,
     "scan_progress": 1
  },
  "output_rows": 81677302,
  "pruning": {
    "partitions_scanned": 168,
     "partitions_total": 291
  }
}
```

Query Profile demo

Poll

1. Query plans operators are

a) Processing statistics

b) filter, join, aggregation, sort

c) Links between nodes

2. How old query plans can you inspect?

a) Only active session

b) 1 day

c) 1 year

3. How to identify the query bottlenecks in query plan?

a) Check the most expensive nodes

b) Check the number of processed rows

c) Table scans are always bottle necks

O'REILLY

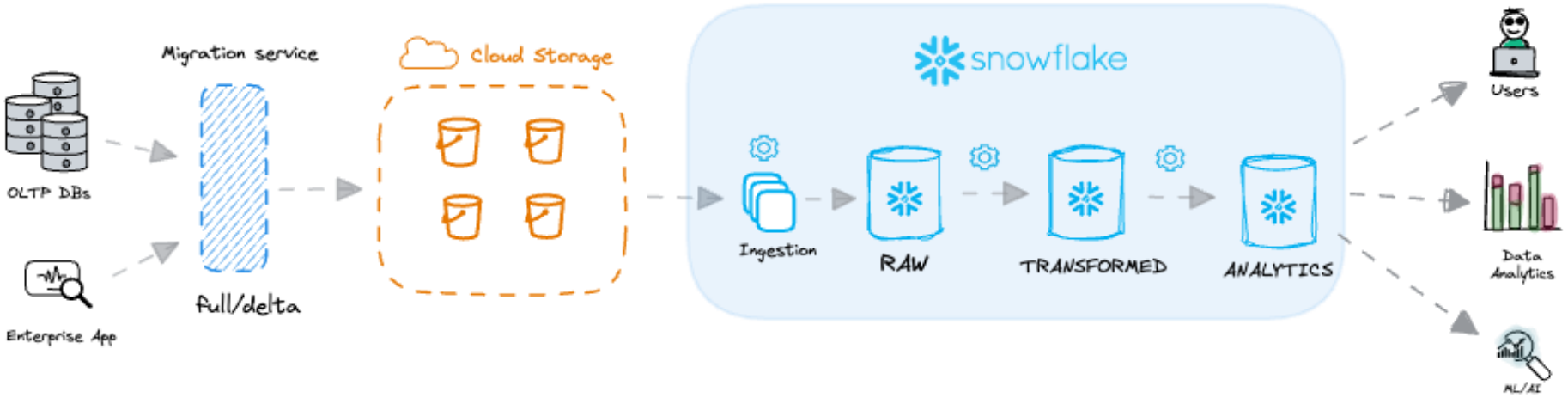
Performance optimization strategies

Tomas Sobotik

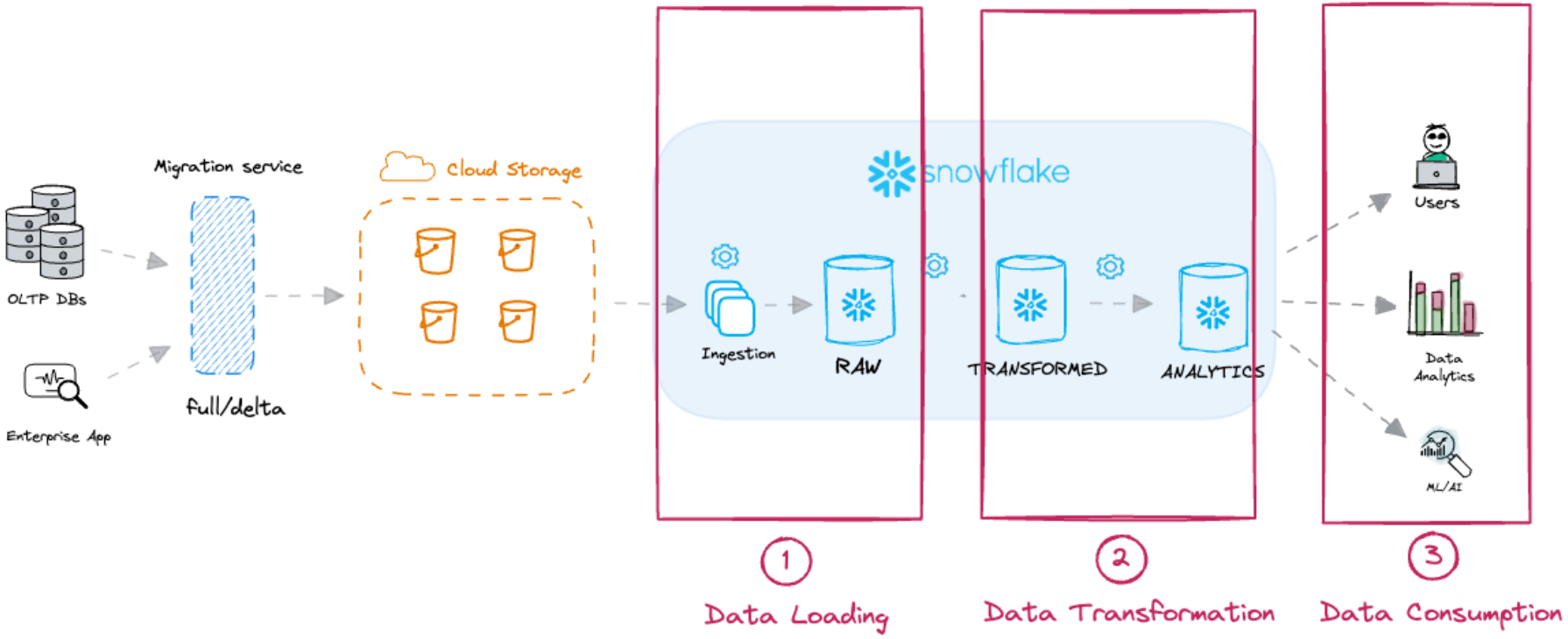
Motivation

- Linking optimization techniques to different areas and categories
- Common techniques and tools – framework around optimizations
- Optimizing performance per category:
  - Different approach per category
  - Different features and techniques per category
- It requires good understanding of Snowflake's architecture and many Snowflake features
  - Complex discipline
  - High level overview followed by deep dives per technique and category

Data pipeline flow and performance optimization strategies



Data pipeline flow and performance optimization strategies



Performance Tuning categories

Data Loading

- Copying data from cloud storage to Snowflake
- COPY, Snowpipe
- Efficient data loading
- Optimizing file size & file organization
- Warehouse utilization efficiency
- Parallel processing

Data Transformation

- Preparing data for consumption
- INSERT, UPDATE, MERGE statements
- Transformation optimizations
- CTEs, JOINs, Agregations

Data Consumption

- End users, dashboards, ML models
- SELECT statements
- Effective data reading
- MP Pruning, filters

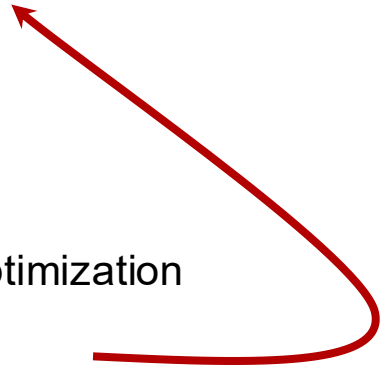
Performance Tuning categories

Data Loading	Data Transformation	Data Consumption
• Copying data from cloud storage to Snowflake • COPY, Snowpipe • Efficient data loading • Optimizing file size & file organization • Warehouse utilization efficiency • Parallel processing	• Preparing data for consumption • INSERT, UPDATE, MERGE statements • Transformation optimizations • CTEs, JOINs, Agregations	• End users, dashboards, ML models • SELECT statements • Effective data reading • MP Pruning, filters

Warehouse configuration Optimization

Performance issue identification

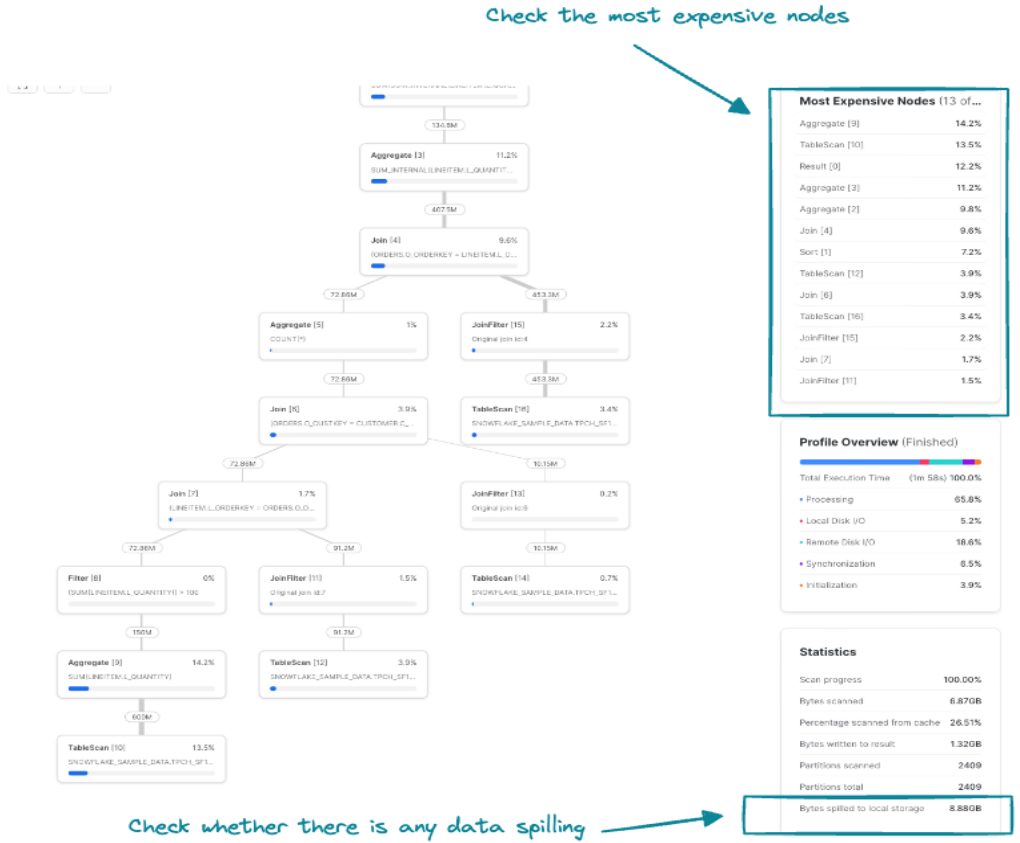
1. Query Profile
   - Current development, maintenance phase
   - Snowsight
   - `GET_QUERY_OPERATOR_STATS()`



2. ACCOUNT_USAGE views
   - Long running project performance optimization
   - Find candidates for optimization first
   - `QUERY_HISTORY`
   - `WAREHOUSE_HISTORY`
   - `TASK_HISTORY`

Identify query bottlenecks

- Check the most expensive nodes
- Check whethere there is any data spilling



ACCOUNT_USAGE views & looking for candidates for optimization

- Typical queries
  - Top n longest-running queries
  - Find long running repeated queries
  - Find a queries with data spilleage

- Performance monitoring
  - Monitor a warehouse load (warehouse utilization metrics soon)
  - Track the average performance of a query over time
  - Create queries categories and organize them by execution time over past month
  - Longest running tasks, SP, ..

- Build a monitoring framework around those queries to continually verified your performance
  - Custom made
  - Many tools on the market

Long running queries

```sql
--top N long running queries
SELECT
    query_id,
    query_text,
    total_elapsed_time / 1000 AS query_execution_time_seconds,
    partitions_scanned,
    partitions_total,
    BYTES_SPILLED_TO_LOCAL_STORAGE
FROM
    snowflake.account_usage.query_history Q
WHERE
    warehouse_name = 'COMPUTE_WH'
    AND TO_DATE(Q.start_time) > DATEADD(day, -30, TO_DATE(CURRENT_TIMESTAMP()))
    AND total_elapsed_time > 0 --only get queries that actually used compute
    AND error_code IS NULL
    AND partitions_scanned IS NOT NULL
ORDER BY
    total_elapsed_time desc
LIMIT
    10;
```

	QUERY_ID	QUERY_TEXT	QUERY_EXECUTION_TIME_SECONDS	PARTITIONS_SCANNED	PARTITIONS_TOTAL	BYTES_SPILLED_TO_LOCAL_STORAGE
1	01b22601-0203-5	select c_name, c_custkey,	118.122000	2409	2409	9536618496
2	01b1c652-0203-4	SELECT NAME as TASK_NAME,	36.130000	11240	255133	0
3	01b1c652-0203-4	SELECT date_trunc('day', CONVE	36.121000	11240	255133	0
4	01b19ec8-0203-4	select date_trunc(?, convert_time	32.376000	674	1020	0
5	01b19ec8-0203-4	select date_trunc(?, convert_time	31.769000	674	1020	0
6	01b2039f-0203-5	select pipe_id, created, d	30.036000	5745	436541	0
7	01b225fb-0203-5	select c_name, c_custkey, o_o	29.269000	2409	2409	556658688
8	01b1dbed-0203-5	CREATE FUNCTION helloFunctic	24.885000	0	0	86064

Full table scan - any filter?

Spilling - bigger WH.
Low hanging fruit

Poll

1. Data Consumption optimization is mainly about

a) Efficient data reading

b) Optimizing file size

c) Warehouse efficiency

2. What indicates you have a small warehouse to handle the query effectively?

a) run time > 5 mins
b) Partition pruning
c) Data spilling

3. Where would you be looking for old queries performance report? Queries ran a month ago

a) `INFORMATION_SCHEMA.QUERY_HISTORY()` table function
b) `ACCOUNT_USAGE.QUERY_HISTORY` view
c) `GET_QUERY_STATS()`

O'REILLY

Common query performance issues

Tomas Sobotik

Common Patterns

- Summarize the common performance issues no matter of query type
- Long Compilation Time
- Long Execution Time
  - Table Scan takes long time
  - Data Spilling and out of memory issue
  - Bad Execution Plan
    - Filters, Joins, Sorts, overall logic

How to solve those issues

Long Compilation times

- What is it?
  - Compilation takes a lot from overall execution time
    - > 50% of total execution time
    - No fixed threshold – set it according yourself
  - It’s varying – not happening every time

- How to find such queries?
  - QUERY_HISTORY
  - Monitor it to find trend

```
Select
  compilation_time,
  execution_time
from
  account_usage.query_history
where
(compilation_time / execution_time) > 0.5
```

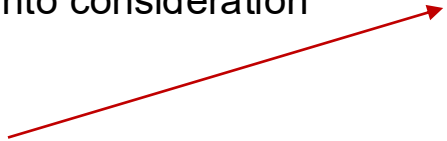
Long Compilation times

What is responsible	How to solve it
• Many small files (partitions) • Many small inserts • A lot of columns • Heavily nested views • Large tables • Expression complexity • Number of expression	• Auto clustering can help with small files ○ Consolidation into bigger files -> better pruning • Limit number of columns (<100) • Limit number of nested views • Simplify the query logic • Limit number of small updates

Long execution times

- What is it?
  - Query takes a longer time to execute compared to previous runs
  - Scanning data in table takes long
  - WH is running out of memory
  - SQL query issues (JOINS, Filters, etc.)

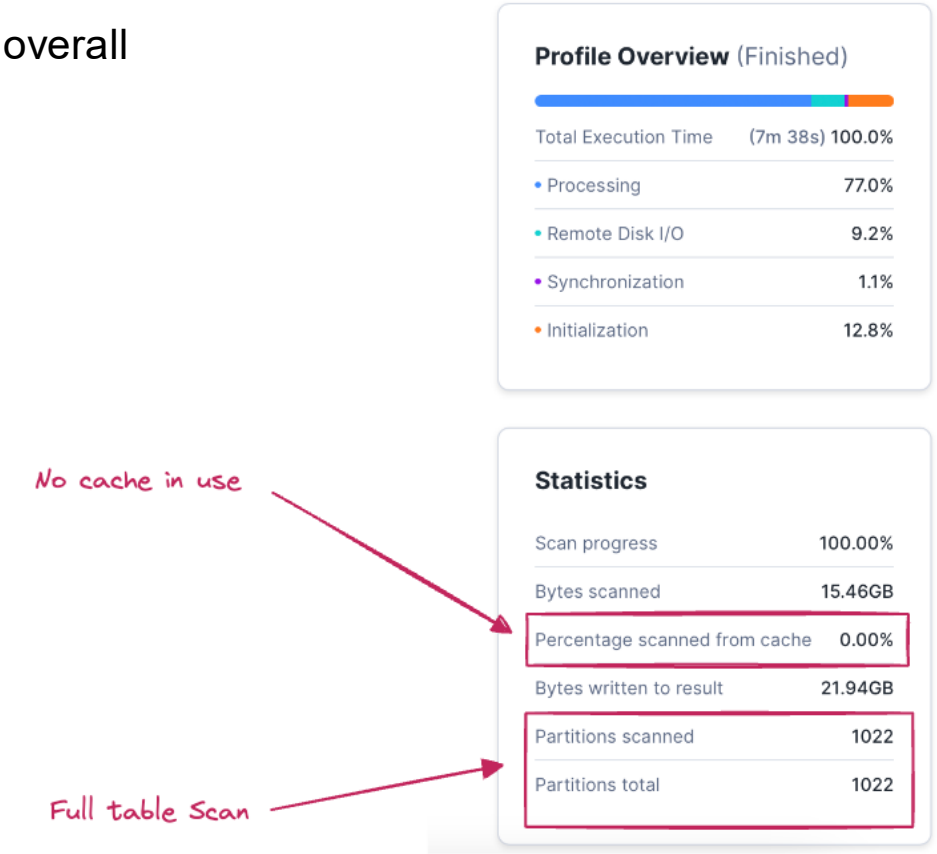
- How to find such queries?
  - QUERY_HISTORY
  - Calculates a query avg run time
    - Do not forget to take into consideration number of records
  - Monitor it to find a trend



```
Select
  …
  execution_time
from
  account_usage.query_history
```

Long Table Scan - identification

- Find queries which scans majority from overall micro partitions
- Include only long running queries
  - Those you want to optimize
- Full table scans
- No cache in use?



Long Table Scans

What is responsible

- A lot of micropartitions on the table
- Filters on non clustered columns
- Range filters (`total_amount between 1000 and 100000`)
- How specific the filters are
- Filter pushdown hasn't been applied
- Expression complexity
- Query result spread over many micropartitions

Filters & clustering

How to solve it

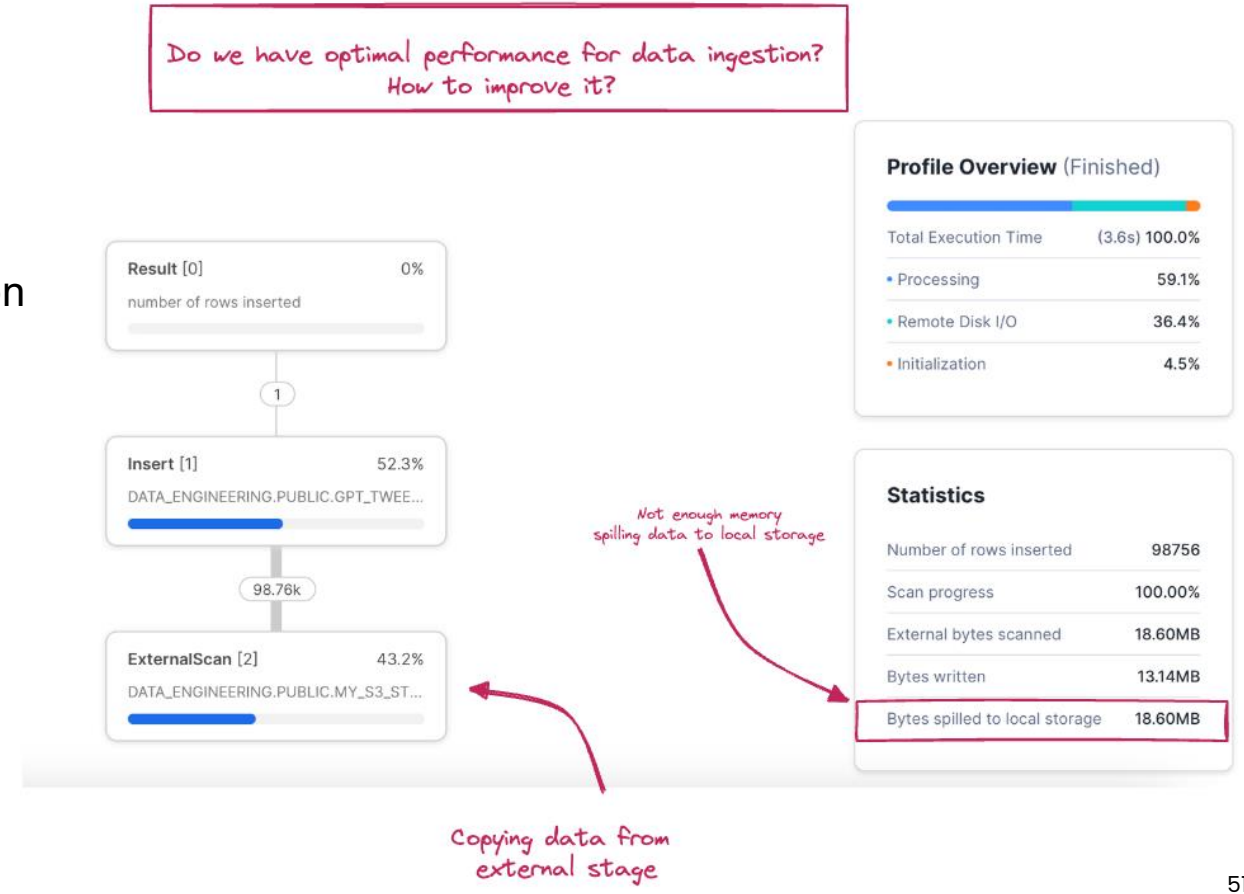
- Use auto clustering on large table
- Organize data along the usual filters
- Add more filters if it is possible
- Simplify the query logic
- Split the query into multiple steps
- Avoid complex expression on the filters – might block the pushdown

Finding queries with long table scans

```
Select
 query_id,
 user_name,
 warehouse_name,
 partitions_scanned,
 partitions_total
from
  account_usage.query_history
where
 partitions_total > 0 and
 (partitions_scanned / partitions_total) > 0.8;
```

Data Spillage

- Not enough memory/CPU power on WH
- Local disk spilling
- Remote disk spilling



Data Spillage

What is responsible

- Large amount of data for processing
- Wrong warehouse size
  - Not fitting the workload
- Concurrency of queries
- Large intermediate results & memory intensive operations
  - Sorting
  - Recursive CTEs
  - Large joins
  - Large aggregations
  - Large window function

How to solve it

- Increase warehouse size (more memory)
- Reduce concurrency
  - Split the workload between more WH
  - Configure multi-cluster Wh
- Improve Pruning
  - Auto clustering
  - Intelligent data insertion
- Simplify the query logic
- Split the query into multiple steps and store the intermediate results

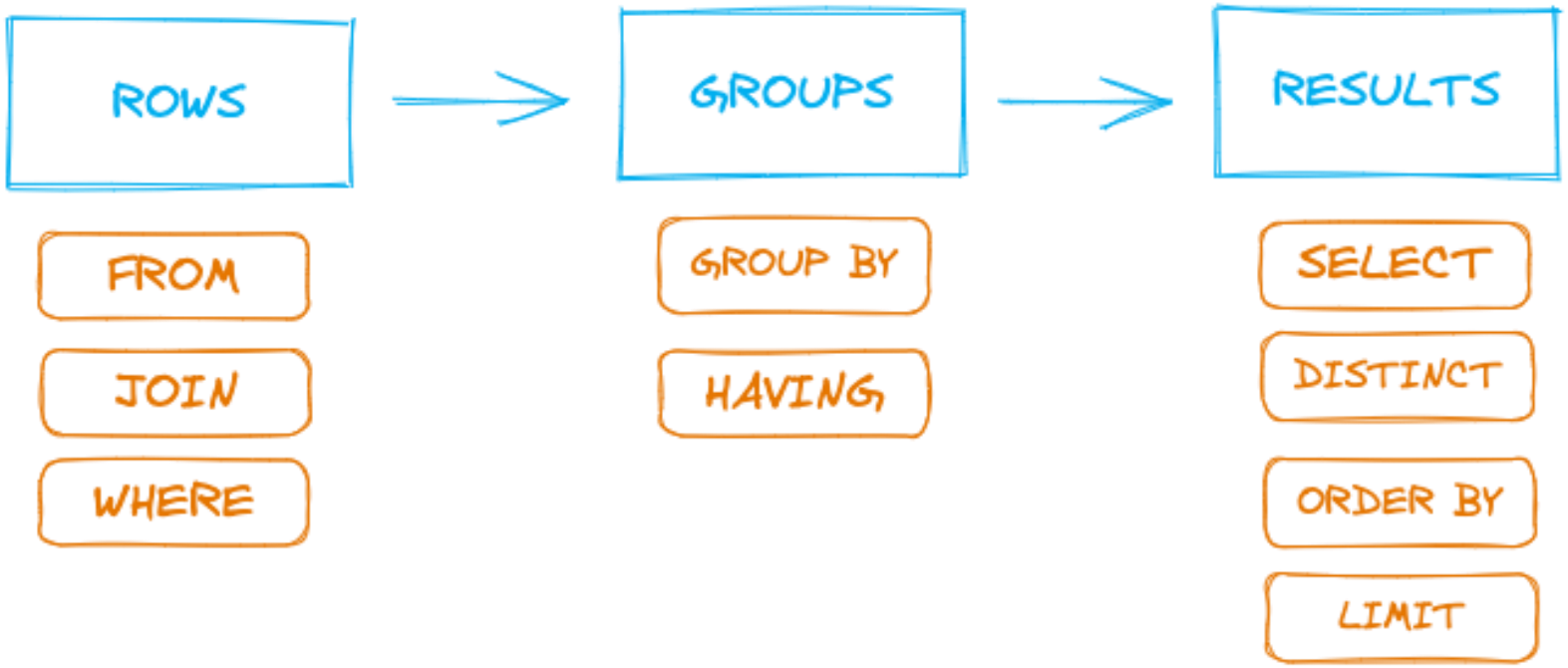
Finding queries with data spilling

```
Select
 query_id,
 user_name,
 warehouse_name,
 bytes_spilled_to_local_storage,
 bytes_spilled_to_remote_storage
from
  account_usage.query_history
where
 bytes_spilled_to_local_storage > 0 or
 bytes_spilled_to_remote_storage > 0;
```

Common best practices per workload

Workload	Best Practice	Avoid
Ad-hoc queries	Monitor it and modify it according found pattern	Not doing anything
ELT / ETL	Load RAW data into stage tables Transform data on top of stage table Sort stage tables according columns used in filters, joins or cluster keys of target table	Not using staging tables Sorting stage tables along different keys than used in filters, joins
Dashboards	Utilize Query result cache Precalculate and preaggregate data	Disabling cache Using UDF or external functions Using functions preventing query result cache
High Concurrency	Use multi-cluster WH + scaling policy Find the right settings for min and max cluster count	Using large WH instead of multicluster Running simple and complex long running queries

Typical DB execution order



Poll

1. You have identified that a lot of queries on top of Sales table have long execution time. Sales table is large table with 5 years of history with 160 columns. Data are clustered based on custom clustering key (order_date, sales_region, sales_agent). How can you improve compilation time?

   a) Remove clustering key

   b) Use bigger warehouse

   c) decrease number of columns

2. Business users have started complaining about long running queries on top of Orders table. After initial analysis you have found out that table scan of Orders table has had decreasing table scan performance. Now it has become too slow. Orders table is clustered on custom clustering key (order_status, shipping_priority). Majority of the queries is related to different time periods, e. g. Orders in last 24 hours, orders in US in May 2023, etc. How can you improve the slow table scans?

   a) Reduce concurrency

   b) Define views on top of Orders table

   c) Update clustering key to include date dimension

Poll

3. You have been asked to propose solution how to decrease cost associated with dashboard queries. Dashboards are built on top of views. Views contains heavy aggregations and data transformations in order to prepare data for consumption in reports. Views are built on top of landing tables. Reports use large REPORT_WH with auto suspend parameter defined to 60 seconds. Mark all performance improvements which you would suggest to implement for lowering the costs:

a) Using larger warehouse – queries will be faster and consume less credits

b) Store intermediate results in standalone tables

c) Disable cache – huge storage cost is associated with using a cache

c) Increase auto_suspend parameter to at least 5 minutes

d) Pre-aggregate data in advance

E) add one more layer with views. First layer will transform data and second layer will aggregate them

O'REILLY

Table Scans & Partition Prunnig

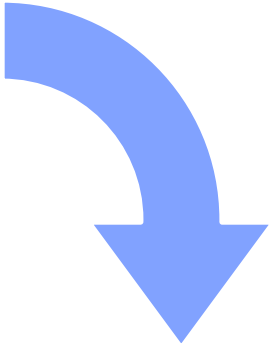
Tomas Sobotik

Micropartitions

Storage unit holding the data

- Physical files holding the logical tables data
- Many micro partitions per table
- Max 16 MB compressed data …50 - 500 MB of uncompressed data
- Immutable
    - Updates create a new micropartition
- Snowflake keeps metadata about micro partitions in service layer
    - Helps with query optimization and performance
- Tables divided horizontally in smaller units

ID	NAME
1	Alice
2	Bob
3	Chuck
4	Jane
5	John
6	Mary



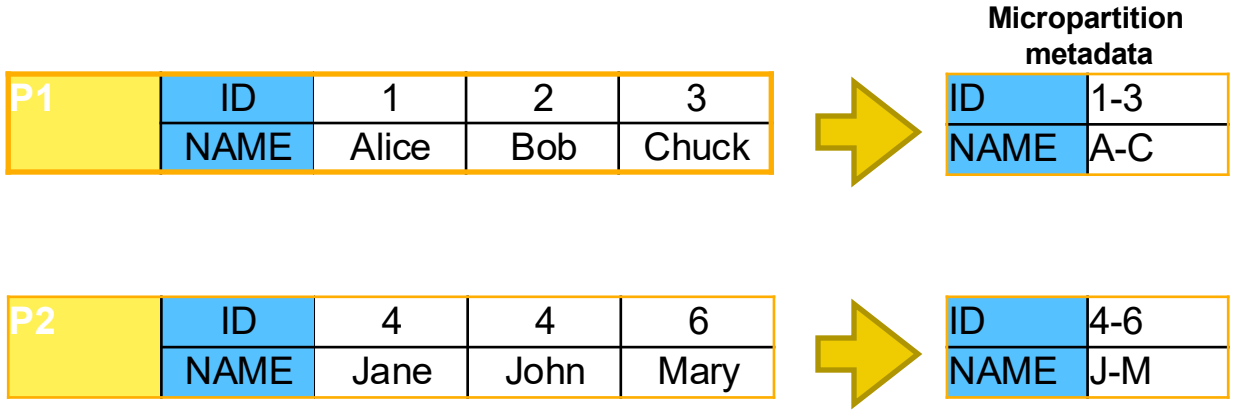
P1	ID	1	2	3
	NAME	Alice	Bob	Chuck

P2	ID	4	5	6
	NAME	Jane	John	Mary

Micropartitions metadata

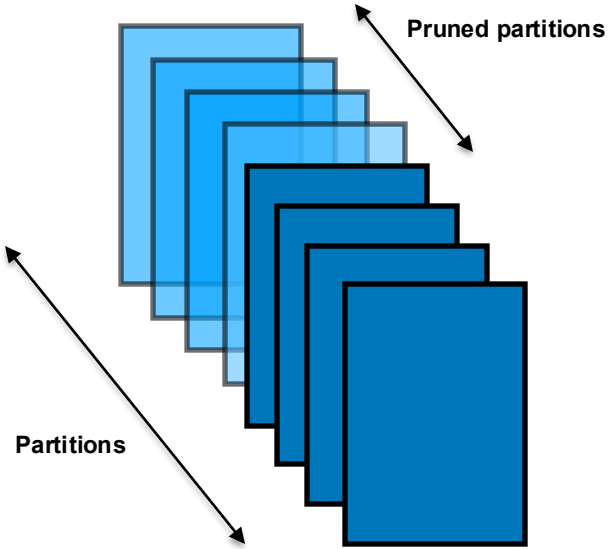
Automatically collected and maintained

- Table level
  - Row count
  - Table size in bytes
  - Referenced files
- Micropartition column level
  - Range of value
  - Number of distinct values
  - Min and MAX values
  - NULL count
- Used by optimizer
- Stored in cloud service layer



Micropartition advantages

- Metadata used by optimizer for partition pruning
  - Horizontal pruning first - by partition
  - Vertical pruning next - by column
- Unit of management for DML - because of immutability
- Thanks to stored metadata some queries are pure metadata operations
  - No need for running virtual warehouse!
- Data Clustering
  - Effective data loading have impact on query performance



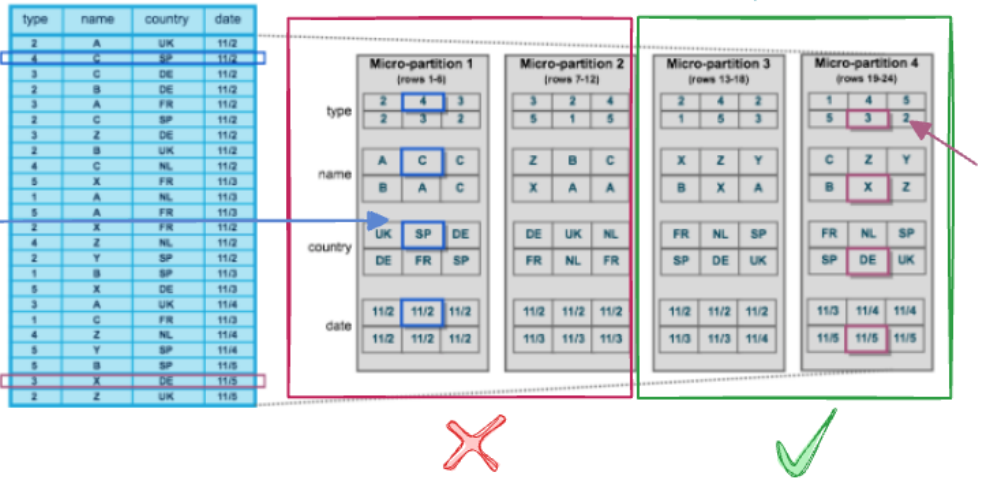
Columnar storage

```
Select name, country from t1 where date ≥ '11/4'
```

Table: t1

Logical Structure

Physical Structure



Data stored by column

- values of each column grouped together and compressed

Hybrid columnar

- Grouping by rows and columns.
- Each micropartition always contains the whole row

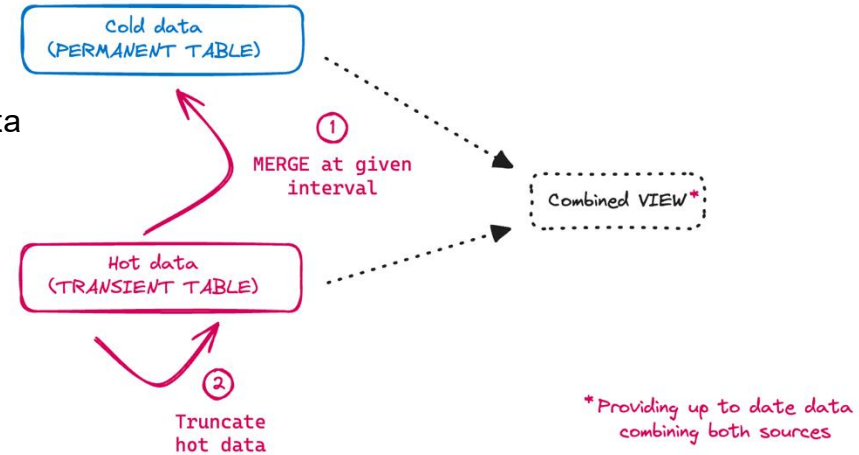
High churn tables and impact on performance

- Single update/delete creates a new micro partition
  - Changed micro partition enters the Continuous Data Protection (Time Travel & Fail Safe) -> additional storage cost
- Micro partition order degrades over time -> performance degradation
- How to identify how churn tables?
  - Ratio of FAIL_SAFE_BYTES divided by ACTIVE_BYTES
    - Columns in TABLE_STORAGE_METRICS view

- How to solve it?
  - Use transient table type for high churn tables
  - Deferred merge

Deferred merge

- A pattern with two copies of data
  - Changed (hot) data or new data for given time interval
    - Transient table
  - Stable (cold) data
    - Hot data are periodically merged into cold data in one batch -> effective MP maintenance
    - Hot data are truncated after the merge
  - A view on top providing unified overview over cold & hot data



- Use cases
  - Real time data with high frequency and lot of updates
  - Dimensions updated every hour
- Good approach when large data needs to be loaded very often

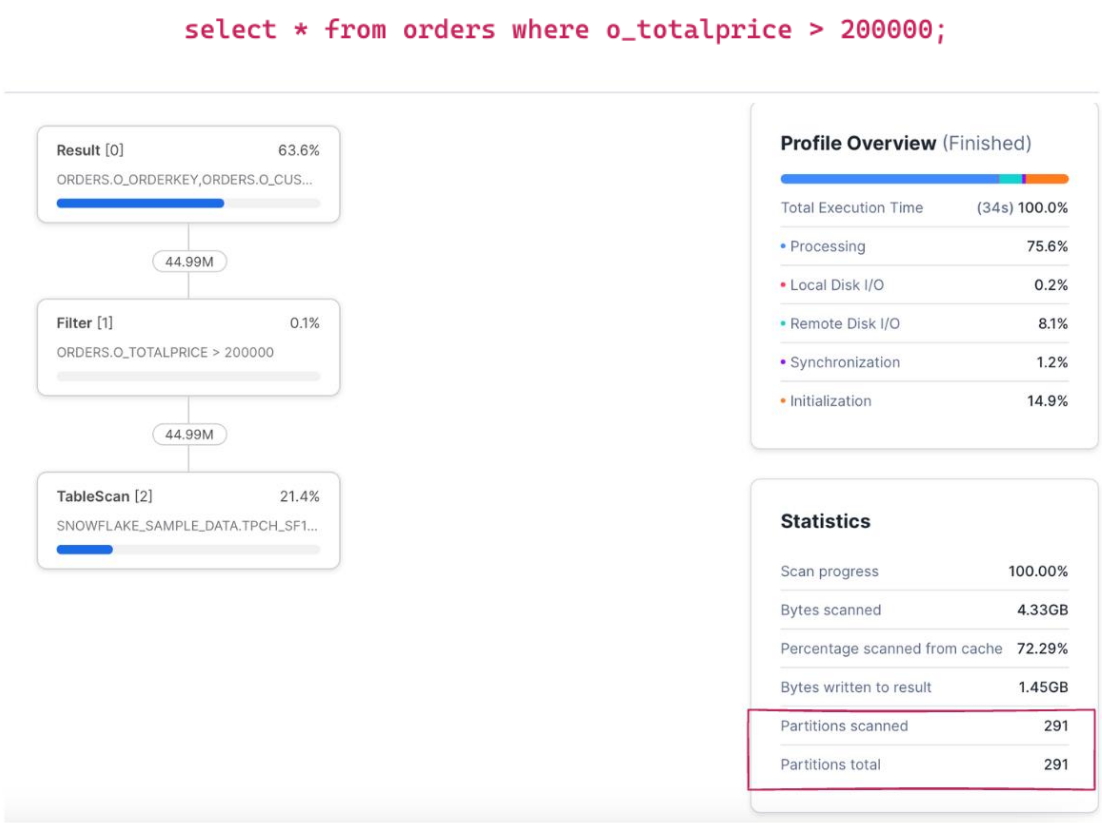
Pruning

- Elimination of not needed micro partitions
- Reading of MP = very costly operation (reading data over network)
- Full table scans are expensive for large tables (many micro partitions)
- Try to avoid scanning a full tables:
  - Limit number of columns - scan only accessed ones
  - Using column metadata to avoid scanning data that is filtered out

- Pruning depends on
    - Applied filters in the query
    - Clustering of data (natural, clustering keys)

Identifying query pruning performance

- Bad pruning
- Full table scan
- Table is not clustered on that column
- Add your data knowledge

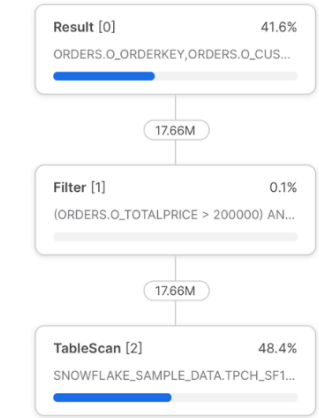


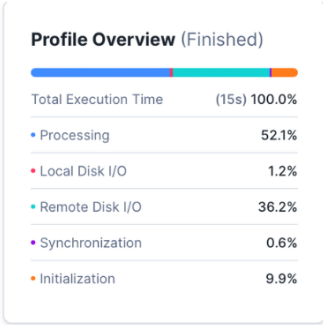
6

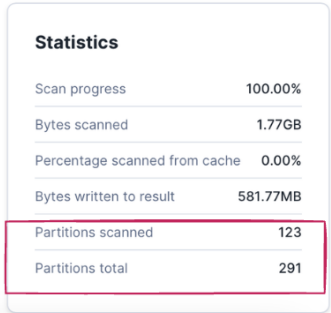
Identifying query pruning performance

```
select * from orders where o_totalprice > 200000 and o_orderdate > '1996-01-01';
```

- Good pruning
- Table organized along date column
- Query using date column could be also written in wrong way and eliminate pruning
- What else can we improve?

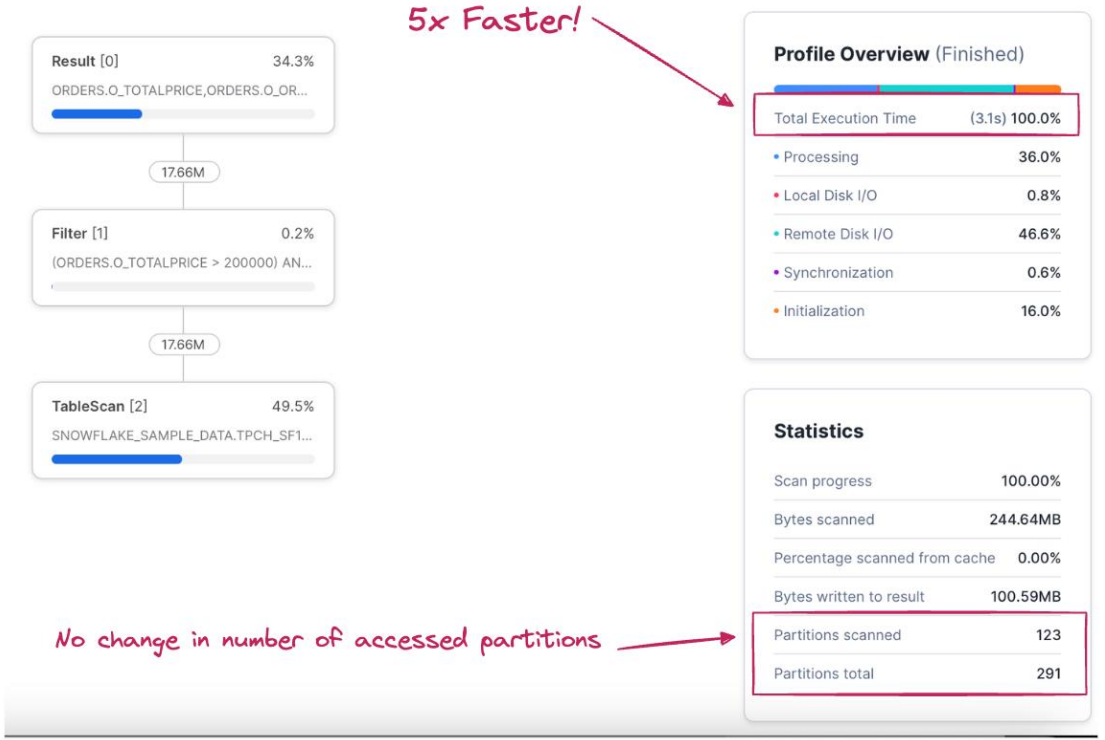






Identifying query pruning performance

- Pruning on column level
- Add only needed/relevant columns



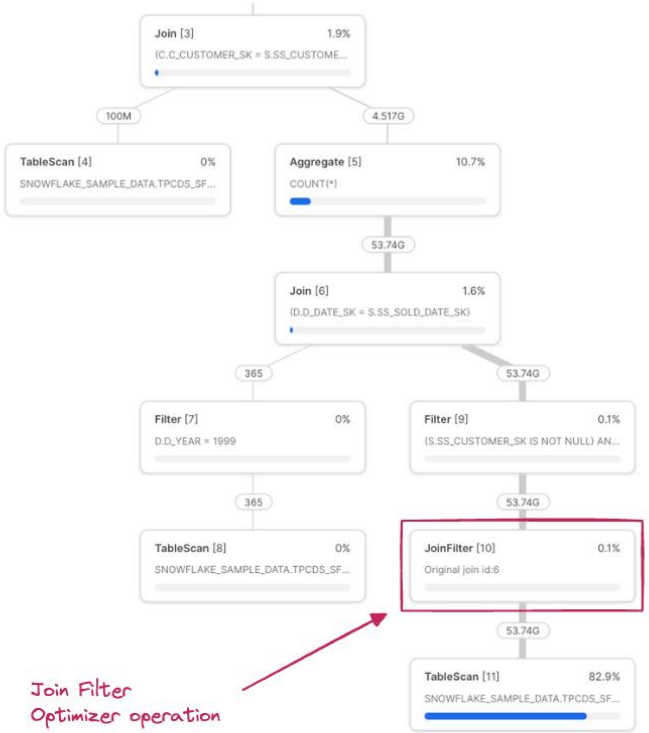

```
select
  o_totalprice,
  o_orderdate,
  o_orderpriority
from orders
where o_totalprice > 200000 and o_orderdate > to_date('1996-01-01', 'YYYY-MM-DD');
```

Join Filters

- Understand how optimizer try to make joins effective
- Happening under the hood
- Join filter = decouple join into two steps
  - Filter step – runs before join and pushed down to table scan
    - Limit number of records on both side as much as possible
  - Join step

- Two techniques
  - Bloom filtering – common standard technique
    - Probabilistic Data structure filters out most rows that would be filtered by the join
    - Depends on data, sometimes nothing could be filtered
  - Range pruning
    - Data structure containing range of the values that could match the join
    - Used to prune micropartitions

Join Filter example



Profile Overview (Finished)

Total Execution Time	(14m 40s)	100.0%
Processing		18.7%
Local Disk I/O		0.4%
Remote Disk I/O		80.3%
Synchronization		0.4%
Initialization		0.2%

Statistics

Scan progress	100.00%
Bytes scanned	123.97GB
Percentage scanned from cache	0.00%
Bytes written to result	0.04MB
Partitions scanned	140635
Partitions total	721729

Improves pruning

```
select
  c.c_last_name, count(*) order_cnt
from
  customer c
join
  store_sales s
on c.c_customer_sk = s.ss_customer_sk
join
   date_dim d
on s.ss_sold_date_sk = d.d_date_sk
where
   d.d_year = 1999
group by
   c_last_name;
```

Exercise

We are going to have a look on queries from slides to find out how micro partition pruning works for some queries and how we can improve it. We will also have a look leveraging micro partition statistics in our queries. Last part of this exercises is dedicated to JOIN filters and how Snowflake optimizer can automatically push down some filters to base tables based on join conditions. This is also something what you should be looking for in query plans and if JOIN filters are not in place think how it can be involved into the query to support partition pruning.

Please find all the queries used in this exercise on **GitHub** *in* `/exersice.sql` *file.*

O'REILLY

Table Clustering

Tomas Sobotik

What is clustering

- Data organization in micropartition
- Crucial element for partition pruning & performance
- Not for all tables
  - Fastest response times required
  - Large tables (TB in size)
  - Cost for maintaining the clustering!
- Cluster key
  - Column, group of columns in table to co-locate the data in same micro-partition

- When to consider defining the cluster key
  - Query performance degraded over time and performance is not acceptable
  - Large clustering depth
  - Need to improve performance
  - Selective queries
  - Queries using sort operation
- To be cost wise = number of queries should be much higher than DML operations (INSERT, UPDATE, DELETE)
- Two types (natural, automatic)

Clustering types

Natural clustering

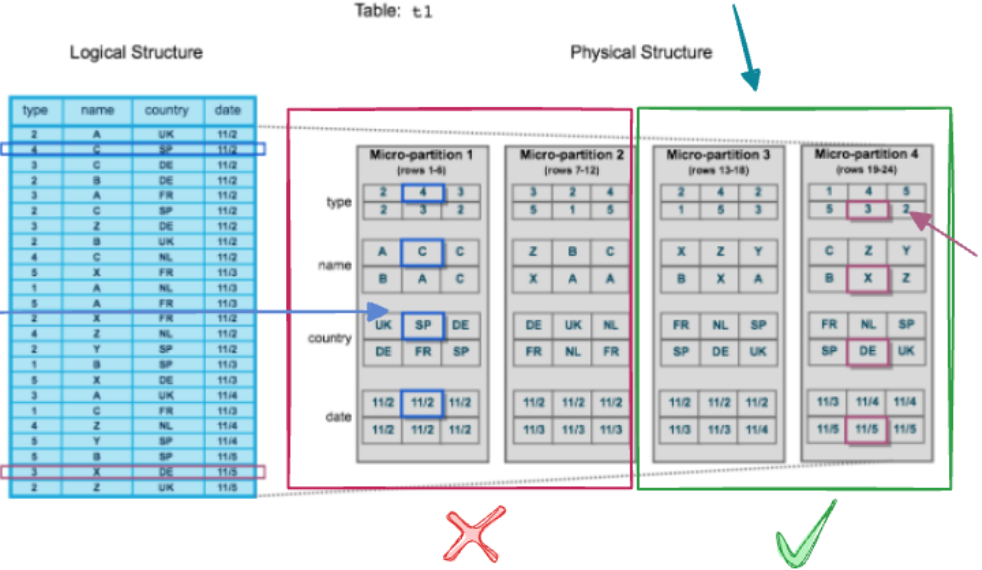
- Manually sorting data when inserting data into tables
- Sorting columns = “clustering key“
- Use columns used in filters, joins, aggregations
- A lot of DML operations can degrade the performance over time
    - manually sort and re-insert the data
- ”free” clustering

Automatic clustering

- Define a cluster key
- Snowflake does the maintenance automatically
- Runs in background on system defined warehouse
- Provide functions to monitor clustering health & cost
- Resume/suspend operations available
- Maintenance consume credits

Natural clustering

```
Select name, country from t1 where date ≥ '11/4'
```



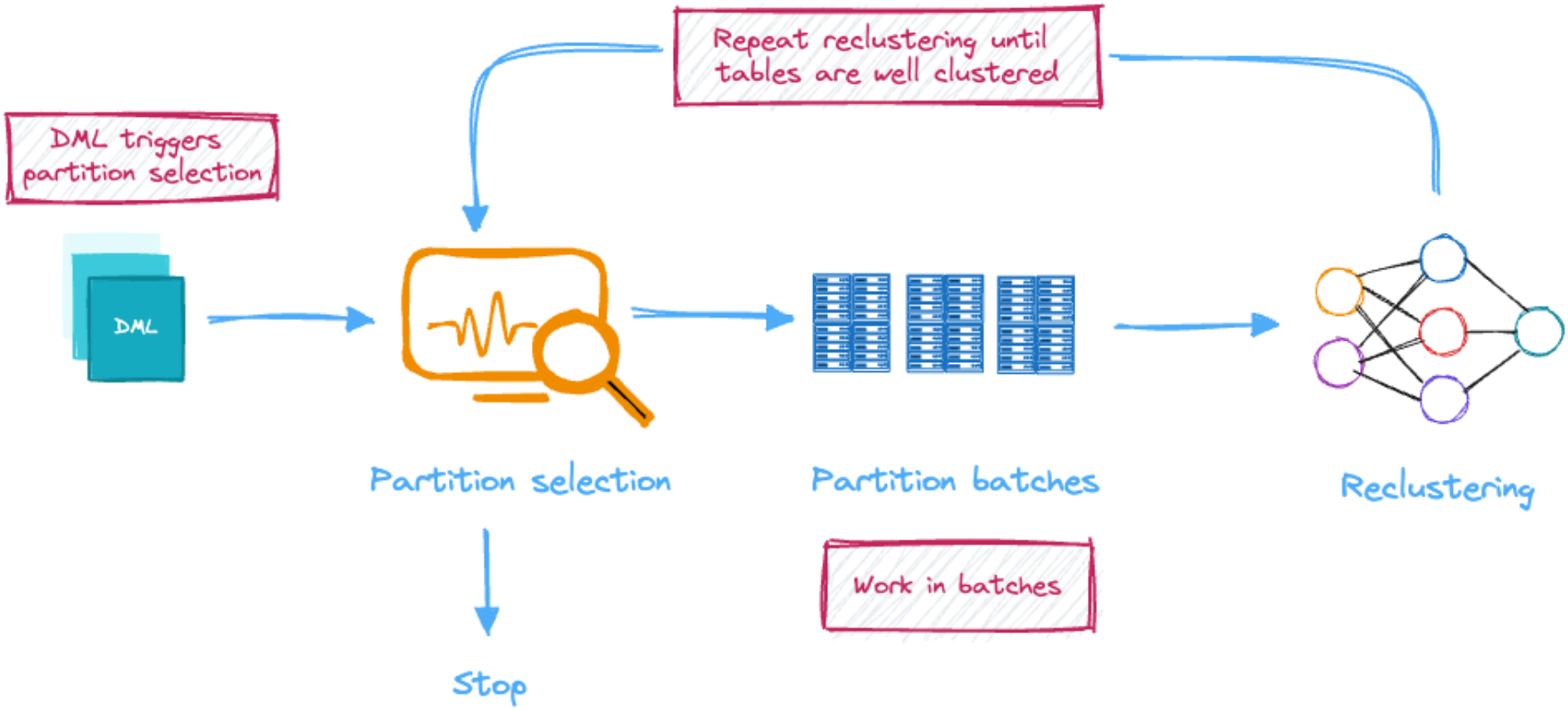
Data stored by column

- values of each column grouped together and compressed

Hybrid columnar

- Grouping by rows and columns.
- Each micropartition always contains the whole row

Automatic clustering workflow

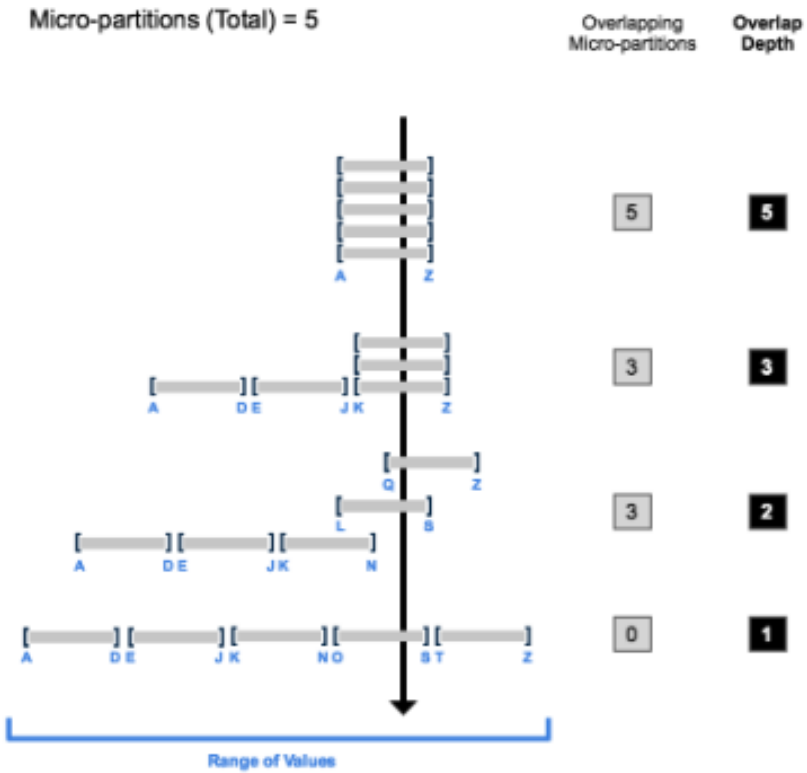


Clustering behind the scenes

Manual Periodic Maintenance	Reclustering is aligned With changes	Incremental clustering
• Full table sort could be expensive and time consuming • Time consuming – blocking other changes • Not a good one on PB tables • Query performance vary	• Keeping DML backlog • Not guaranteed to have good clustering metrics	• Background services continuously improving clustering • Does not interfere with other DML • You can change clustering key if you need • Goal: achieve the best pruning possible

Clustering depth and overlaping

- Show how well the table is clustered
- **Goal:** smallest depth – better clustered table
- Can be used for monitoring of the clustering health
- Empty table – clustering depth – 0
- Constant state = can't be improved further



System functions for clustering monitoring

- **`SYSTEM$CLUSTERING_DEPTH`**
  - Computes average depth of the table based on input columns
  - Can be used to estime how clustering could improve the pruning

- **`SYSTEM$CLUSTERING_INFORMATION`**
  - Returns clustering information including the depth, depth_histogram, number of micropartitions, average overlap, total_constant_partition_count etc.

- **`ALTER TABLE SUSPEND/RESUME RECLUSTER`**

- To check a cost
  - View **AUTOMATIC_CLUSTERING_HISTORY**
  - **AUTOMATIC_CLUSTERING** warehouse – Snowflake's provided warehouse for reclustering purposes

Automatic clustering developer workflow

1. Validate if it is necessary or beneficial to do
    - Query performance degrades over time
    - Execution times are not sufficient
2. Define a testing queries to evaluate the benefits
    - Typical queries which currently struggle and using planned clustering keys
3. Enable automatic clustering
4. Check the clustering health to ensure you have used the right clustering keys
    - SYSTEM$CLUSTERING_INFORMATION
5. Do the validation
    - Have been response time improved?
    - Is it sufficient or worth of the cost?
6. Disable clustering/ change clustering key if not
7. Monitor clustering cost and queries performance

Exercise

During this exercise we are going to work with system functions related to clustering. We will try to check how good is defined clustering and whether we can improve it by changing the clustering key.

We will also try the natural clustering and its impact on query performance.

Please find all the queries used in this exercise on **GitHub** *in* `/exersice.sql` *file.*

O'REILLY

Effective Data Loading

Tomas Sobotik

Motivation

- Data ingestion is essential part of every project
- Easy to do, everyone does it
- Do you do it effectively?
  - Don’t you waste your compute resources?
  - Do you use the right tool with optimal setup? (COPY, Snowpipe, Task, etc.)

- Many best practises
  - File size & organization
  - Parallelization
  - Warehouse utilization
  - Frequency

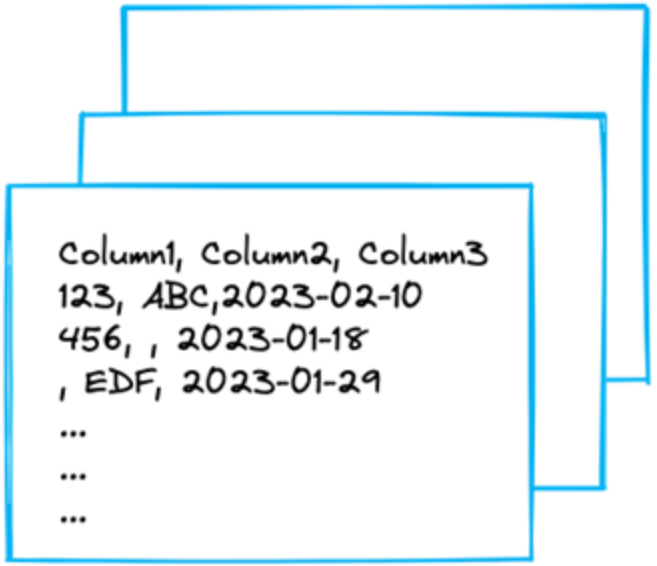
- By following the best practises you can significantly improve the performance and lower a cost related to data ingestion

File size & organization

Organize files by path

- Create logical paths per required dimension
  - Time
  - Location
  - Department

```
US/east_cost/finance/monthly/2022/02
```



File sizes

- Split large files before loading
- Batch / combine small files before loading
- Recommended size is 100 to 250 MB (compressed)

File locations

All files in single location

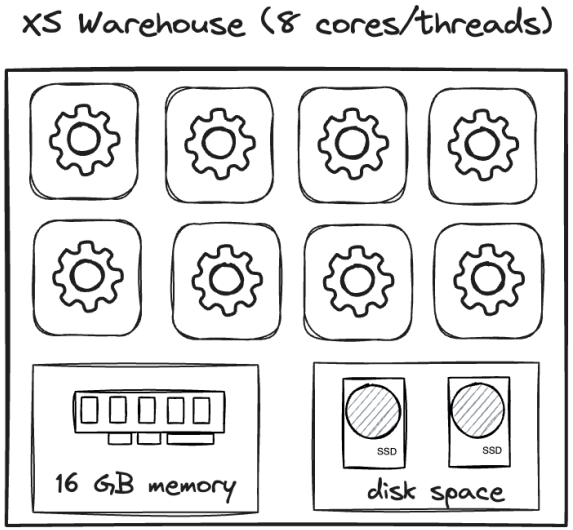
Processing is slower as engine needs to scan more files to find the right ones

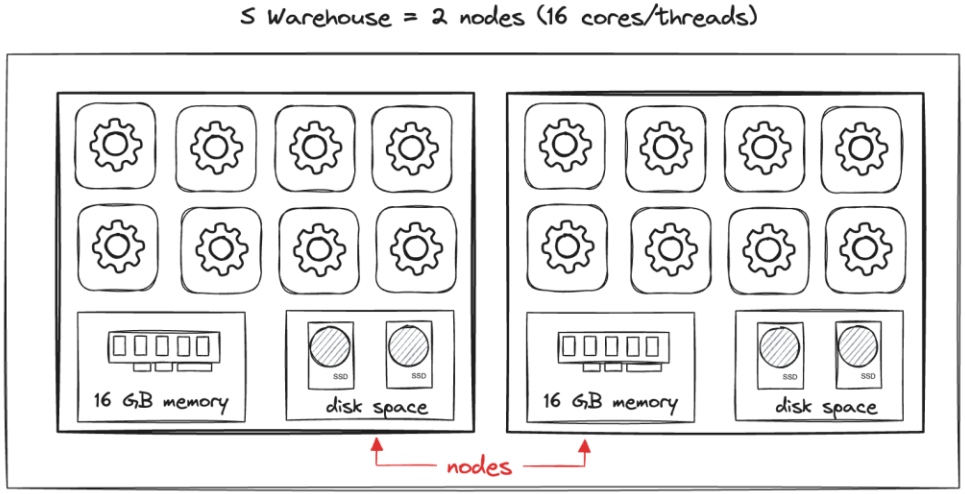
Organised files in multiple locations

Faster. Multiple directories with same file types allow for scanning fewer files

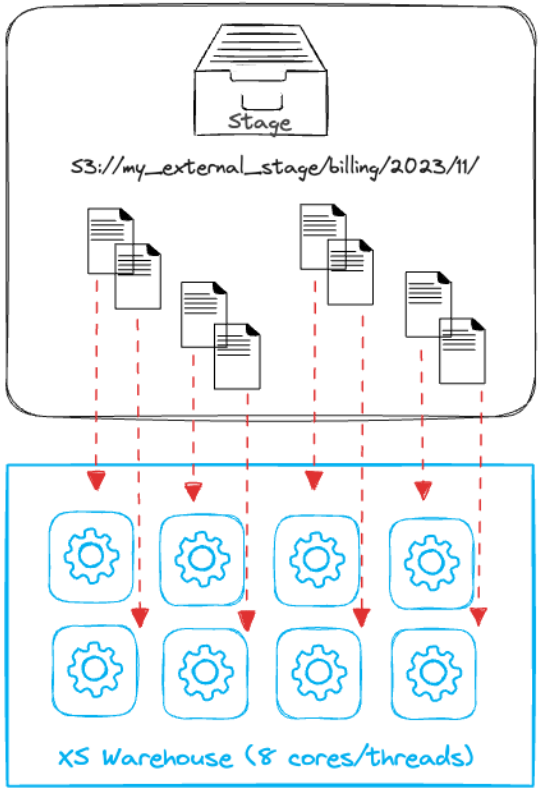
Warehouse utilization

- Node
  - EC2 instance equivalent
  - 8 cores/threads
  - 16 GB of memory
  - Disk space (SSD)



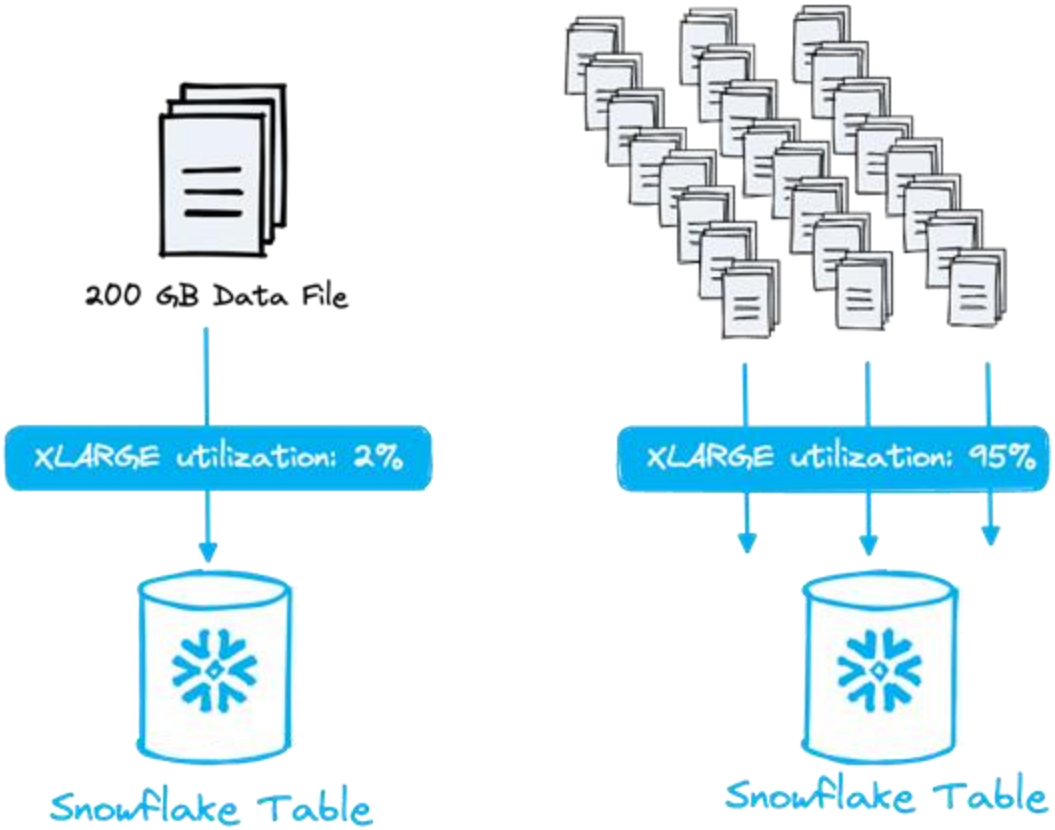


Warehouse utilization



Warehouse size	Number of Cores/threads	Number of processed files in parallel
XS	8	8
S	16	16
M	32	32
L	64	64

Serial vs Parallel processing



Takeaways

- You need to keep the most warehouse threads busy (warehouse utilization)
- Files needs to be properly sized and compressed
- Use PATTERN filter in COPY command
- Files needs to be organized
- Use the smallest warehouse possible
  - Little queuing is better than unutilized cores
  - Increasing the size until meet your SLA
  - Remember you are always billed for first 60 seconds

- Consider the reload frequency
  - What is the accepted latency compared to cost?

Ingesting 1 GB of data in 100 MB chunks (10 files):

Warehouse size	Number of cores	Utilization ratio	Run time	Billed time	Cost/day	Cost/year
Medium	32	~30%	10s	60s	$0.2	$73
Extra small	8	~100%	50s	60s	$0.05	$18.25

Ingestion method selection

COPY (task)

- Properly sized files
- Capacity planning
- Need to keep warehouse busy for at least 60 secs (8 files)
- On commit latency
- Only SQL knowledge

COPY (serverless task)

- Properly sized files
- No capacity planning
- Can offer better warehouse utilization
- Better suited for ingestions when data files amount is varying
- Always better choice if run time < 40s
- On commit latency
- Only SQL knowledge

Snowpipe

- Properly sized files
- Perform poorly on many small files (utilization cost)
- No capacity planning (serverless)
- Suited for delta loads
- 1.25x regular compute
- Latency in minutes
- Easier adoption to changing ingestion needs
- Automation
- SQL & cloud computing concept knowledge

Poll

1. What is recommended file size when ingesting data into Snowflake?

a) 16 MB
b) 100 – 250 MB
c) 1 GB

2. How many files can process in parallel the smallest virtual warehouse (XS) which has 1 node?

a) 1
b) 8
c) 16

3. Which scenario will provide best performance/price ratio when ingesting 2.5 GB in total with 250 MB per file?

a) XS Warehouse
b) L Warehouse
c) XL Warehouse

O'REILLY

How to improve your queries

Tomas Sobotik

Motivation

- Backed up with knowledge from this course, let’s try to optimize some SQL constructs
- Data reading/consumption optimizations
- Snowflake optimizer is being constantly improved and does a lot of work automatically
- By following some tips we can help to optimizer to be more effective
  - WHERE clause, SORT and aggregations
  - JOIN Orders and explosions
  - MERGE statement
  - Effective CTE using

Optimizing WHERE clause

- Don’t convert dates to char for comparison
- It blocks partition pruning!

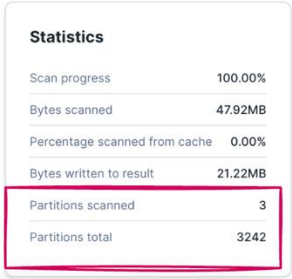
Run time: 42s

```
select * from snowflake_sample_data.tpch_sf1000.orders
where to_char(o_orderdate, 'DD-MM-YYYY') = '06-11-1994';
```

Some pruning has been done.

Run time: 3s

```
select * from snowflake_sample_data.tpch_sf1000.orders
where o_orderdate = to_date('06-11-1994','DD-MM-YYYY');
```



Best possible pruning!

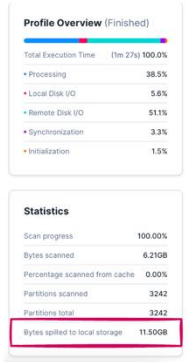
Optimizing SORT clause

- Most expensive operation – full table scan
- Remove unnecessary sort operation from inner queries
- Expensive and useless operation
- Use approx function to find out uniquness, etc. (large datasets)
- Other approx functions for percentile, top_k, etc.

Run time: 87s

```
select count(distinct o_orderkey)
from snowflake_sample_data.tpch_sf1000.orders;
```

1.5 billions

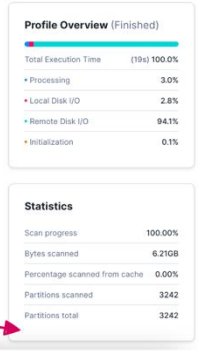


Local spilling

Run time: 19s

```
select approx_count_distinct(o_orderkey)
from snowflake_sample_data.tpch_sf1000.orders;
```

1.49 billions

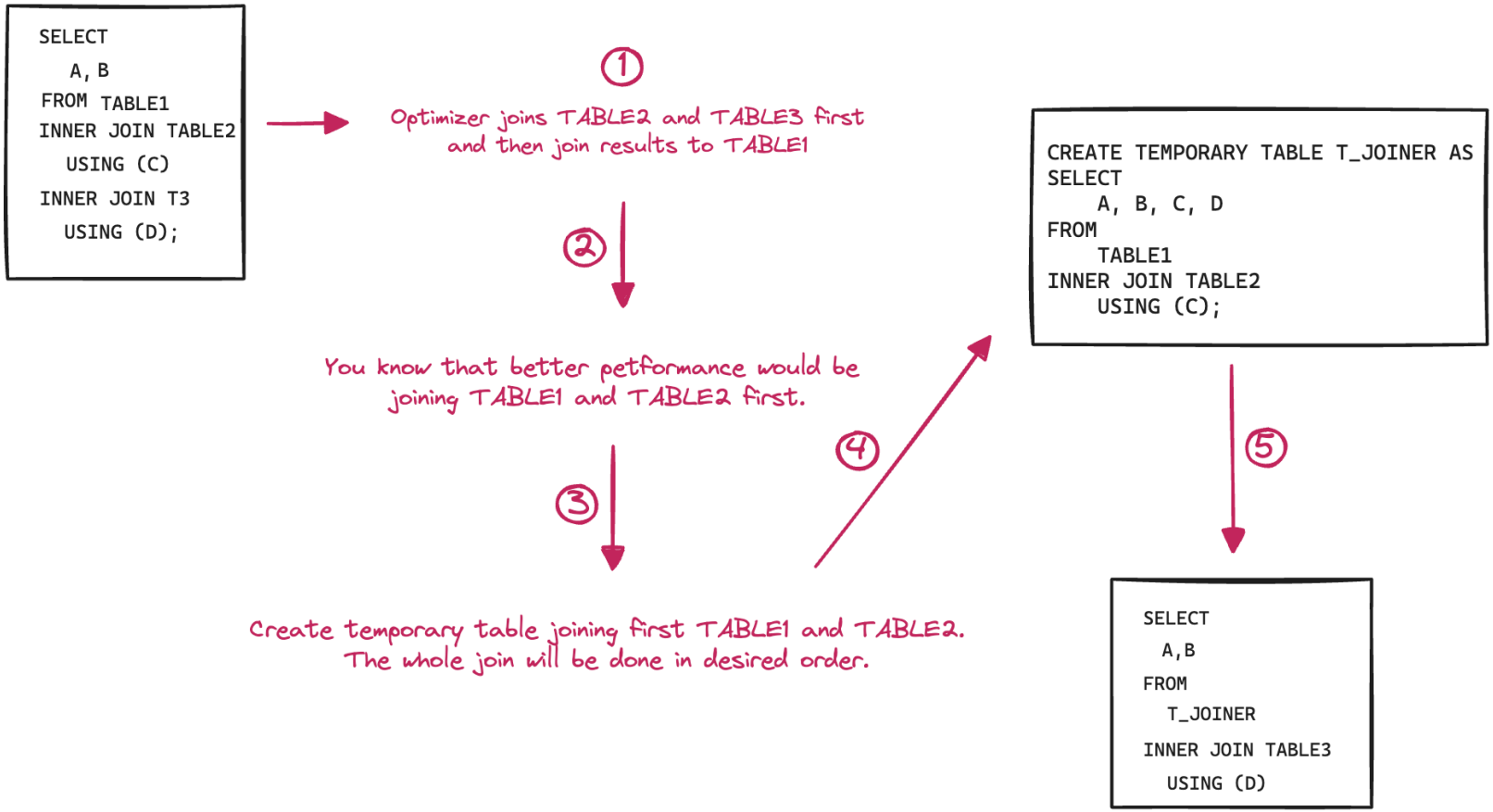


No Spilling!

JOIN Order

- Optimizer tries to find best possible order for joining tables
- You have valuable knowledge of data
- Sometimes you can suggest the better order
- When order might be wrong
  - Left side of the join contains much more data then right side
  - The most of the time from query run time is spent on join operation
- Effective order: (small + medium) + big
- How to identify the join order? -> Query Profile
- Snowflake favors small (dimensions) table on the left side of the join (build side) and big fact tables on right side (probe side)

JOIN Order – solution with temporary table



JOIN Order - analysis

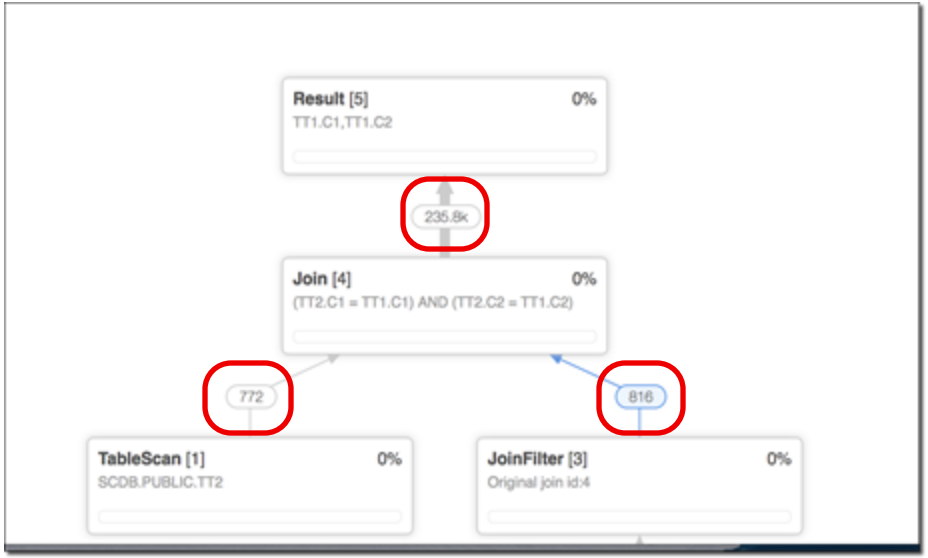
- Right handed join tree is much better for optimizer
- Better parallelism
- Notice the small tables on left side
- Efficient use of memory

JOIN Explosion

- Join operator result produce more rows than the larger side of join has

- Issues
  - Spillage & Out of memory
  - Poor performance

- Causes
  - Cartesian join
  - Joining on non unique keys
  - No join filter / wrong filter
- Fixes
  - Add more filters if possible
  - Add join key/change join key to be unique
  - Join with small tables first
  - Rewrite a query
    - to force pushdown
    - Use temp table for intermediate result



Effective usage of CTE

- Named SQL query, could be referenced multiple times
- Defined using `WITH` clause
- Queried by using name
- Multiple CTEs per query – comma delimited
- Benefits
  - Code readability - easier debugging because of isolated subqueries
  - Procedural logic in SQL (top to bottom)
  - Single definition, used multiple times

```
with my_cte as (
  select 1
  from tableA
),
my_cte_2 as (
  select 2
  from tableB
)
select *
from my_cte
left join my_cte_2
```

Using CTE effectively

- If CTE is referenced only once -> optimizer handle it like normal query -> no impact on performance

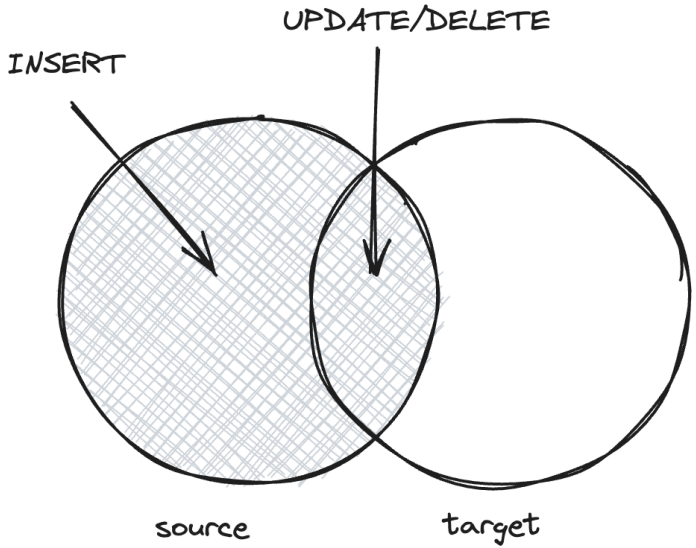
- Multiple reference of same CTE
  - Sometimes is faster repeat the logic, sometimes reusing same CTE
  - Depends on complexity of the query, more complex -> reusing CTE
  - Need to experiment

- Column pruning
  - Single reference of CTE -> everything works as expected
  - Reusing same multiple times -> blocking column level pruning
  - Repeat a CTE logic to get column level pruning
  - List only required columns in CTEs to enable the column level pruning -> especially for tables with a lot of columns

- Let’s have a look on examples

Optimizing MERGE statement

- Upsert – insert, updates, deletes in single SQL statement
- Real time data, incremental processing – important concept to perform it in optimized way



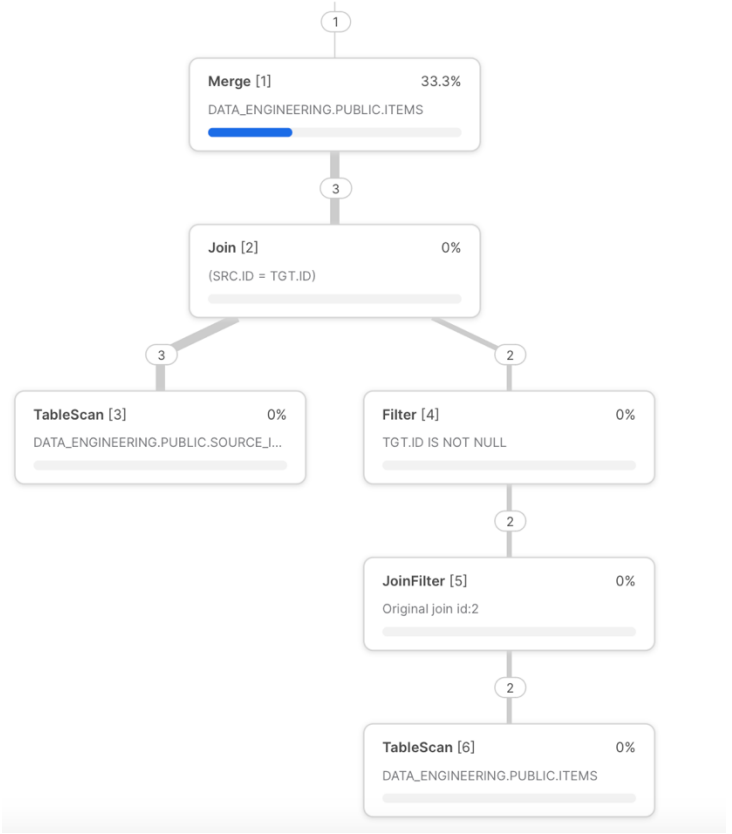

```
MERGE INTO target USING
source ON target.key = source.key

WHEN MATCHED THEN
UPDATE SET target.value = source.value

WHEN NOT MATCHED THEN
INSERT  (key, value) VALUES (source.key, source.value)
```

Optimizing MERGE statement

- Scanning target table
  - Include filter to enable pruning, use well clustered column (not an id)
- Source and target tables are joined
  - Wrong JOIN order – manual control
  - Use unique keys for join (exploding joins)
- Use dynamic pruning to improve performance
  - Additional join filters to support partition pruning as join condition



Exercise

During this exercise we will focus on trying the explained concepts on real queries. We are going to try optimize the WHERE clause. We will have a look how could be improved the MERGE statements and how CTEs works in relation to query performance.

Please find all the queries used in this exercise on **GitHub** *in* `/exersice.sql` *file.*

Summary

- Common performance issues and how to solve them
- Strategies for performance optimization
- Proficient Data Loading
- Effective Virtual Warehouse configuration
- Query Life Cycle
- Table clustering and micro partition pruning
- Optimizing often used SQL constructs

Summary

- Common performance issues and how to solve them
- Strategies for performance optimization
- Proficient Data Loading
- Effective Virtual Warehouse configuration
- Query Life Cycle
- Table clustering and micro partition pruning
- Optimizing often used SQL constructs

Good Luck with your optimizations!

Summary

- Common performance issues and how to solve them
- Strategies for performance optimization
- Proficient Data Loading
- Effective Virtual Warehouse configuration
- Query Life Cycle
- Table clustering and micro partition pruning
- Optimizing often used SQL constructs

Good Luck with your optimizations!

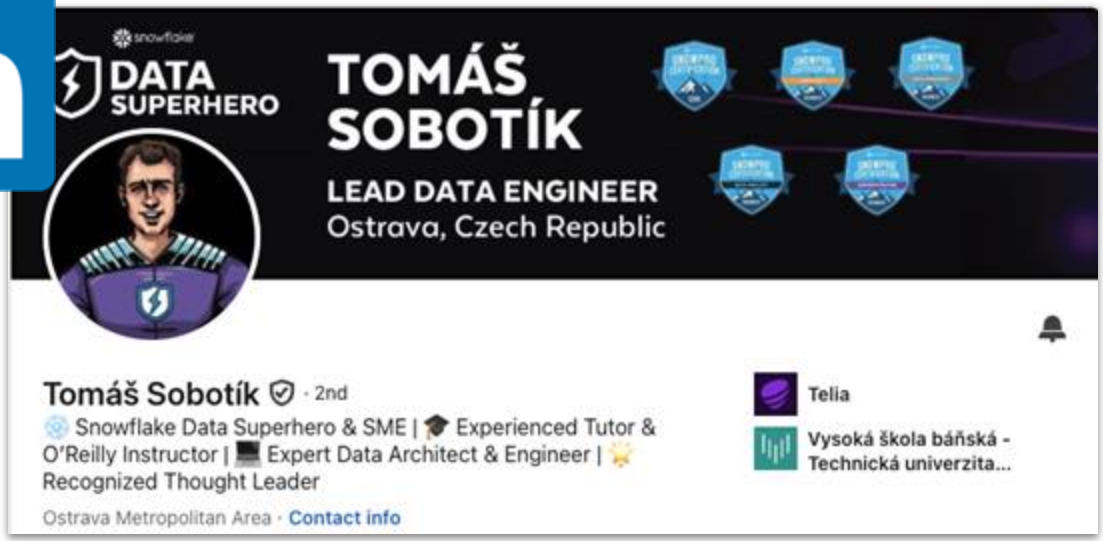
Please, share your feedback.

Summary

- Common performance issues and how to solve them
- Strategies for performance optimization
- Proficient Data Loading
- Effective Virtual Warehouse configuration
- Query Life Cycle
- Table clustering and micro partition pruning
- Optimizing often used SQL constructs

Good Luck with your optimizations!

Where to Find Me



https://www.linkedin.com/in/tomas-sobotik/

Where to Find Me

https://tomas-sobotik.medium.com/

Live Courses

Your Instructor

Tomáš Sobotík

SnowPro Core Certification Bootcamp

Published by O'Reilly Media, Inc.

Beginner to intermediate

Snowflake Fundamentals Bootcamp

Published by O'Reilly Media, Inc.

Beginner

Snowflake for Data Engineering Bootcamp

Published by O'Reilly Media, Inc.

Intermediate

Snowflake Administration

Published by O'Reilly Media, Inc.

Intermediate

Optimizing Snowflake

Published by O'Reilly Media, Inc.

Intermediate

On demand Courses

Your Instructor

Tomáš Sobotík

On-demand course

+ Add to playlist

SnowPro Core Certification Course: Understand the Core Foundations for the Exam

By Tomáš Sobotík

★★★★★ 1 4h 4m

O'Reilly Media, Inc. • June 2025

Quiz

CC in German, English + 6 more

This on-demand course reviews the fundamentals of the Snowflake platform and helps learners prepare for the SnowPro Core certification exam. Through a mix of concise presentations and demonstrations, the course covers architectural concepts of the platform and best practices for building scalable data solution...

Thank you!